

PRT582 ASSIGNMENT 2

Group One

Group Members

Ben Carruth
Gia An Vong
Sandipkumar Gangajaliya
Zhe Wei

Table of Contents

Abstract:.....	4
Introduction:	4
Objectives:	4
Tasks:.....	4
Scope:.....	5
Requirements and User Stories:	5
User Stories	5
Librarian/Staff	5
Functional Requirements (Staff/Librarian)	5
User Stories.....	5
Non-Functional Requirements:.....	8
Usability:	8
Accuracy:	8
Availability:.....	8
Maintainability:	8
Software/system Requirements:	8
Team Composition and Task Allocation:.....	9
Non-Functional Testing:.....	10
Usability Test.....	10
Moderated and in-person usability testing	10
Lab usability testing:	10
Guerrilla testing:	10
Moderated and remote usability testing	10
Phone interviews:	10
Card sorting:.....	11
Unmoderated and remote usability testing	11
Session recordings:	11
Online testing tools and platforms:	11
Unmoderated and in-person usability testing.....	11
Different software tools that we can use for usability testing:	11
First-click Test:	12
Results:.....	15
Results:.....	21

Five-second Test:	23
Results:	26
Performance Test:	28
Load testing:	28
Stress testing:	28
Scalability testing:	28
Breakpoint testing:	28
Endurance testing:	28
Spike testing:	28
Web LOAD:	28
Load Ninja:	28
Load View:	29
Apache JMeter:	29
Micro Focus LoadRunner:	29
Rational Performance Tester:	29
Neo Load:	29
Loadster:	29
Page Speed:	29
Python timeit:	29
Creating performance test using JMeter:	29
Running the test:	34
Test for staff login page using cookies and cache:	39
Security Test:	39
Types of Software Security Testing:	40
Software Security Testing tools:	40
Functional Testing	45
Test plan:	45
Test strategy:	45
Features to be tested:	45
Features not to be tested:	46
Test deliverables and schedule:	46
Responsibilities:	46
Comparison and justification of automation tool	48
Results and reports:	52

Results and reports:	54
Results and reports:	57
Results and reports:	58
Results and reports:	60
Results and reports:	61
Results and reports:	63
Results and reports:	64
Conclusions	66
Ethical consideration:.....	66
Further work:	67
Lesson learned:	67
Conclusion:.....	67
Appendix	67

PRT582 Software Group Project

Abstract:

The purpose of this unit and assignment two is to develop the skills required for designing and developing structured software solutions for the testing environment of open-source applications. Analysis of both problems associated with code and the techniques for solution testing. Finally, a critical evaluation of all ethical considerations faced in development.

Introduction:

The Library Management System is an online program used by librarians to manage members, books, book lending, and penalties. Consequently, a suitable testing strategy that employs innovative technological tools and a skilled automated testing team is essential for delivering a perfect product to end customers.

Objectives:

The aim is to create a testing plan that outlines all activities and responsibilities, communication mechanisms, a bug-tracking system, acceptance criteria, and deliverables for each testing phase.

Tasks:

The testing team must verify that all executable features function as specified in this functional testing scope. In addition, it is vital to provide input on how the system might be enhanced to provide a better user experience.

The primary tasks would be defined as below:

Task ID	User story	Acceptance criteria
1	Allow the staff/librarian to Login or Signup.	- System allows creating a new user - Registered users can log in to the system
2	Can search for books through book id	- Books are filtered by input book id
3	Can add, remove, and manage the books.	- A new book can be added to the database - Users able to delete a book - Users can edit the book info
4	Can add new students upon applications.	- A new student can be added to the database
5	Can issue books for students.	- The librarian can issue a book to a specific student.
6	Can view issued books with details like student information, book name, book's category, issued date, return date, fine, etc.	- Book-issued records can be shown on the system
7	Can view registered students and enable search them by their id.	- Student info is displayed on a list - Student data are searched by student id

8	Can entry for any book that is returned.	- The returned book's status must be changed to "not-issued"
9	The system calculates the fine for overdue books on their return.	- Cannot be tested automatically and doesn't work in manual testing either

Scope:

All published features on the web applications, including:

Data input and validation

Data display on the graphic user interface (GUI)

Screen transitions and layout

Multiple browsers in defined scope: Chrome, Firefox, Microsoft Edge, Safari (macOS only)

Requirements and User Stories:

User Stories

The application Library management System chosen, only one user Librarian/Staff are included. The functional requirements are focused on librarian roles. The non-functional and system requirements and demonstrated a use case diagram below.

Librarian/Staff

The librarian registers new students, adds and maintains the books, collects fines for overdue books, and issues books to students who need them.

Functional Requirements (Staff/Librarian)

1. Allow the staff/librarian to Login or Register.
2. Search for books through book_Id
3. C.R.U.D. the books database.
4. Add new students upon applications.
5. Issue books for students.
6. View issued books with loan details like student information, book name, book's category, date of loan, return date, fine applicable.
7. View registered students and enable to search them by their id.
8. Returning information for any book received from loan expiry.
9. The system calculates the fine for overdue books on their return.

User Stories

STORY ID: (LMS-001 - LMS-007)

STORY ID: LMS-001	STORY TITLE: Joe Blow	
USER STORY:		IMPORTANCE:
AS A: CDU Student		2 / 5
I WANT: Borrow a book and return it within the allowed time.		ESTIMATE:
SO THAT: I do not attract a fine from the library.		2 / 5
ACCEPTANCE CRITERIA		TYPE:

AND I KNOW I AM DONE WHEN:		SEARCH
I get confirmation of return via email.		WORKFLOW
		MANAGE DATA
		PAYMENT
		REPORT / VIEW ✓

Requirement & user story for Librarian Roles

STORY ID: LMS-002	STORY TITLE:	
USER STORY:		IMPORTANCE:
AS A: library staff (librarian)		5 / 5
I WANT: to create staff account		ESTIMATE:
SO THAT: I can access to the application to monitor book, student and book borrowing and returning activities.		3 / 5
ACCEPTANCE CRITERIA		TYPE:
AND I KNOW I AM DONE WHEN:		SEARCH
I can login to the system with my username and password		WORKFLOW
		MANAGE DATA ✓
		PAYMENT
		REPORT / VIEW

STORY ID: LMS-003	STORY TITLE:	
USER STORY:		IMPORTANCE:
AS A: library staff (librarian)		4 / 5
I WANT: add a new book information		ESTIMATE:
SO THAT: I can manage the book issuing and returning status		2 / 5
ACCEPTANCE CRITERIA		TYPE:
AND I KNOW I AM DONE WHEN:		SEARCH
I can see the added new book from the dashboard screen		WORKFLOW
		MANAGE DATA ✓
		PAYMENT
		REPORT / VIEW ✓

STORY ID: LMS-004	STORY TITLE:	
USER STORY:		IMPORTANCE:
AS A: library staff (librarian)		4 / 5
I WANT: add a new student		ESTIMATE:
SO THAT: I can manage student data		2 / 5
ACCEPTANCE CRITERIA		TYPE:
AND I KNOW I AM DONE WHEN:		SEARCH
I can see the new student from student list screen		WORKFLOW
		MANAGE DATA ✓

		PAYMENT
		REPORT / VIEW ✓

STORY ID: LMS-005	STORY TITLE:	
USER STORY:		IMPORTANCE:
AS A: library staff (librarian)		4 / 5
I WANT: issue a book		ESTIMATE:
SO THAT: I can manage that the student who borrowing which book and any fine for the late return		2 / 5
ACCEPTANCE CRITERIA		TYPE:
AND I KNOW I AM DONE WHEN:		SEARCH
I can see the student with his borrowing books.		WORKFLOW
		MANAGE DATA ✓
		PAYMENT
		REPORT / VIEW ✓

STORY ID: LMS-006	STORY TITLE:	
USER STORY:		IMPORTANCE:
AS A: library staff (librarian)		4 / 5
I WANT: return a book from student		ESTIMATE:
SO THAT: I update the returning date and the book is ready to borrow		2 / 5
ACCEPTANCE CRITERIA		TYPE:
AND I KNOW I AM DONE WHEN:		SEARCH
I can see the issue record with student info, book id and returning date, including fine amount		WORKFLOW
		MANAGE DATA ✓
		PAYMENT ✓
		REPORT / VIEW ✓

Requirement & user story for Librarian

STORY ID: LMS-007	STORY TITLE:	
USER STORY:		IMPORTANCE:
AS A: library staff (librarian)		3 / 5
I WANT: check an issued book status		ESTIMATE:
SO THAT: I update the status of the book is returned or not		3 / 5
ACCEPTANCE CRITERIA		TYPE:
AND I KNOW I AM DONE WHEN:		SEARCH
I can see the issue book record about return date and time.		WORKFLOW
		MANAGE DATA ✓

		PAYMENT
		REPORT / VIEW ✓

Non-Functional Requirements:

Usability:

The user interface should be simple enough for everyone to understand and get relevant information without any special training.

Accuracy:

The data stored about the books and the fines calculated should be correct, consistent, and dependable.

Availability:

The System should be available for the duration when the library operates and must be recovered within an hour or less if it fails.

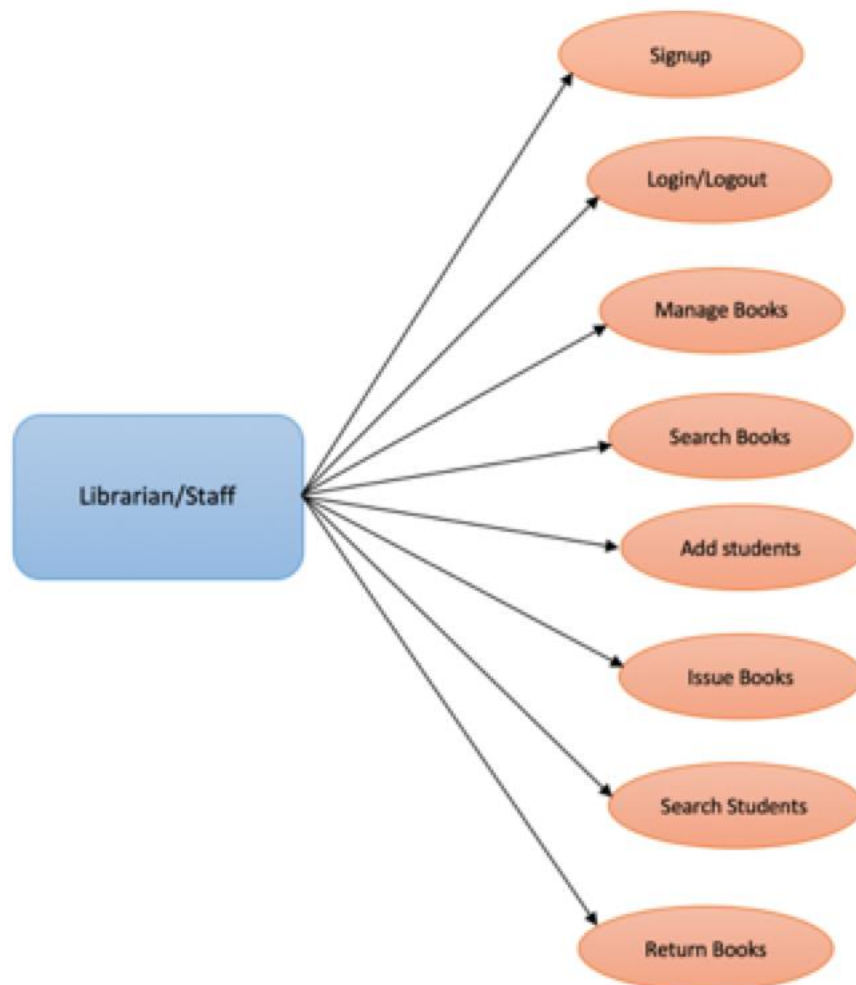
Maintainability:

The software should be easily maintainable and adding new features and making changes to the software must be as simple as possible. In addition to this, the software must also be portable.

Software/system Requirements:

1. The system should be platform independent, can run on Windows Server/Linux based OS.
2. Backend language python.
3. Front-end framework Django
4. Relational DBMS like MySQL or SQLite

Use Case Diagram



Team Composition and Task Allocation:

The report, presentation and assignment have been divided by all group members. The main components and their assignment group member is as follows:

Report, documentation, PowerPoint Presentation: Ben Carruth

Non-Functional Testing: Md Al Amin Sarker / Gia An Vong

Functional Testing: Gia An Vong

Test Cases and Results: Sandipkumar Gangajaliya / Zhe Wei

Conclusion: Gia An Vong

Non-Functional Test Results: Ben Carruth

Non-Functional Testing:

Non-functional testing is a type of software testing that verifies how software works and how well it works on non-functional parameters such as usability, performance, security, availability, maintainability, reliability, and accountability of the software. It is just as important as functional testing because it plays a crucial role in customer satisfaction, ensures that our product is working as it should be, and checks things not covered in functional tests.

Usability Test

Usability testing refers to the user interface that should be simple enough for representative users to understand and get relevant information without special training.

Usability testing done too late in the design phase is a typical mistake. If we wait until the very last minute before our product is delivered, we will not have the time or resources to address any problems, and we will have wasted a great deal of time building our product the wrong way.

This testing cycle is completed when:

1. The product is ready.
2. Customers are asked to evaluate the product.
3. If any further changes are found.
4. The product is returned to the development team with feedback for changes required.
5. The product requires usability testing.
6. No changes are required.
7. The product is launched into the market.

Several types of usability testing are available, these include:

Moderated and in-person usability testing

Tests that are moderated and conducted in-person offer the most control. They require many resources, but they are great for gathering comprehensive data.

Lab usability testing:

This kind of usability testing is conducted inside a lab that has been specially designed for it. While a skilled moderator watches and observes, test participants work on computers or mobile devices to accomplish tasks.

Guerrilla testing:

In guerrilla testing, test subjects are randomly selected from a crowded area, typically a coffee shop, shopping centre, or airport. They are frequently requested to do a brief usability test in return for a gift card or another incentive.

Moderated and remote usability testing

A skilled moderator must conduct remote and moderated usability tests using a computer or phone. They allow you to choose from various testers while utilizing a moderator's expertise and depth.

Phone interviews:

In a phone usability test, the user's online activity is remotely recorded. At the same time, the moderator gives participants verbal instructions on how to accomplish tasks on their computers and

gathers comments.

Card sorting:

Concepts are written on digital note cards, and then participants organize the cards into groups and categories. After sorting the cards, they discuss their reasoning in a debriefing session led by the moderator.

Unmoderated and remote usability testing

These passive testing techniques, which rely on computer programs, provided insights into how people interact with a website in its "natural environment."

Session recordings:

Software captures user behaviour on a website, including mouse clicks, movements, and scrolling.

Online testing tools and platforms:

Numerous online testing tools enable us to observe user activity on your website from a distance. Some of these platforms let us pay participants to take quick tests, while others monitor actual users' actions as they navigate our website.

5-second test:

This is a simple method for gathering qualitative information on users' initial reactions to your website.

First-click test:

First click testing helps gather information on user expectations and determine where the best places to put menus and buttons.

Unmoderated card sorting:

Users can take the test whenever it is convenient, and this method is quicker and less expensive than moderated sorting.

Unmoderated and in-person usability testing

Unmoderated in-person tests are administered in a controlled physical environment without a test administrator.

Observation:

Researchers can observe participants' facial expressions and body language during observational testing without the moderator's involvement.

Eye-tracking: Using a specialized pupil-tracking gadget attached to a computer, researchers track and analyse users' eye movements during eye-tracking examinations. The computer can produce heatmaps or movement pathway diagrams by examining where users focus their attention when asked to perform a job.

Different software tools that we can use for usability testing:

1. First click and Five Second Test by UsabilityHub. <https://usabilityhub.com>)
2. Google Analytics. <https://marketingplatform.google.com/about/analytics/>
3. Google Forms. <https://docs.google.com/forms>

4. Google Mobile Friendly Tests. <https://search.google.com/test/mobile-friendly>
5. Usability Test using Nibbler. <https://nibbler.insites.com/>

For our library management system software, we have chosen the “*Unmoderated and remote*” methodology to perform usability testing that relies on computer programs. These passive testing techniques offer perception into how visitors engage with a website in their "natural environment." And we will create two types of unmoderated and remote usability testing for users that are staff or librarians in our case. The first is the First click test, and the second is the 5-second test.

First-click Test:

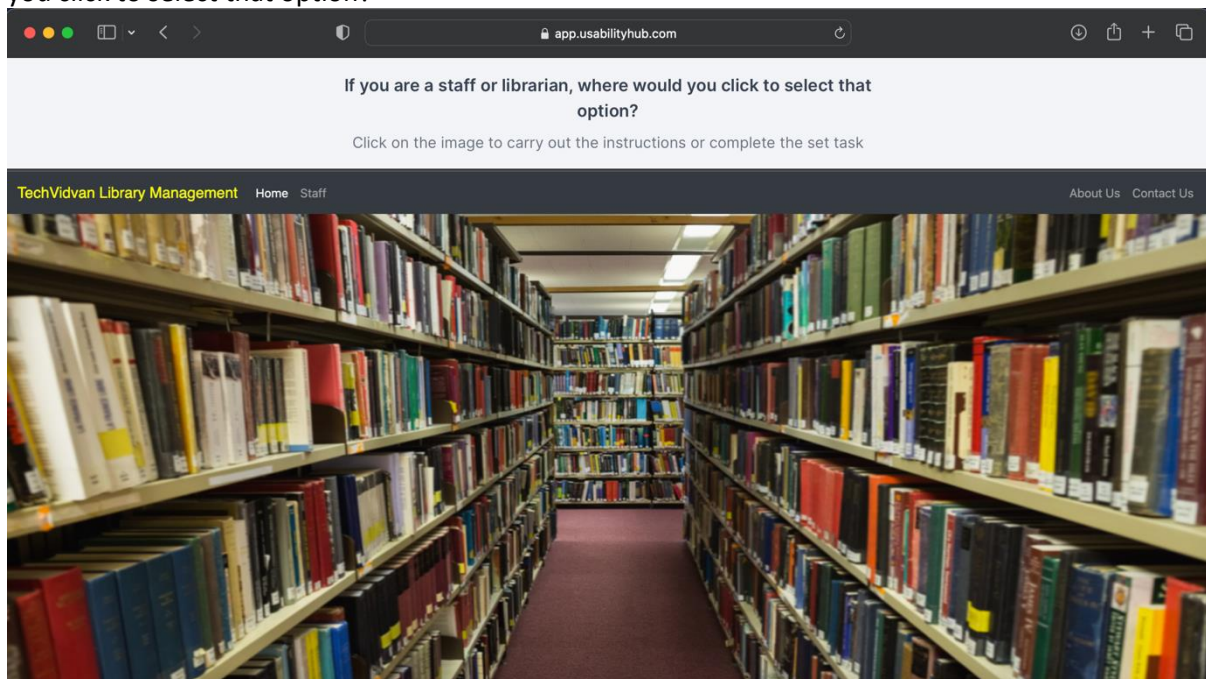
First-click testing aims to determine whether users can quickly identify the location they must navigate to finish a task. The software tracks the participant's mouse movement as they answer questions like "Where would you click to buy this product?". We created different first-click test cases for our software in Usability Hub (<https://usabilityhub.com>), a user research and usability testing platform. In the free version of UsabilityHub, we can only create two test cases on a single link.

We designed a usability test case for the home page using first-click method and generated a shareable test link to get user feedback.

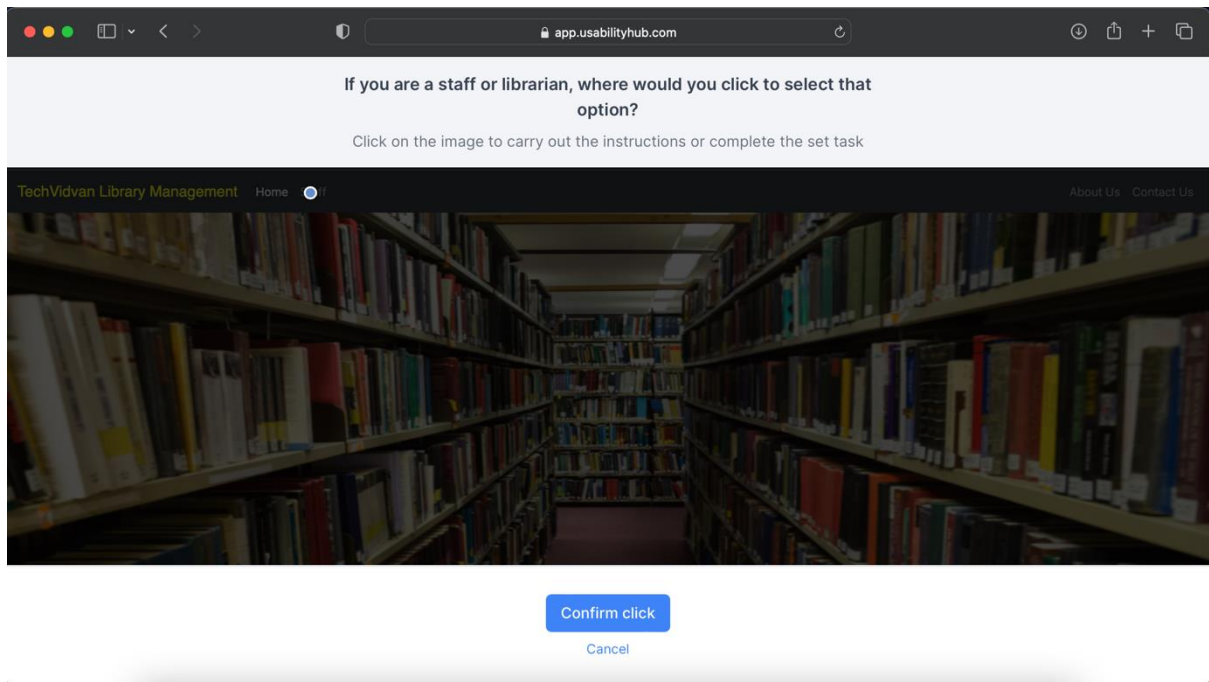
Test link: <https://app.usabilityhub.com/do/f57983ac9a28/d949>

Below are the steps to take the tests:

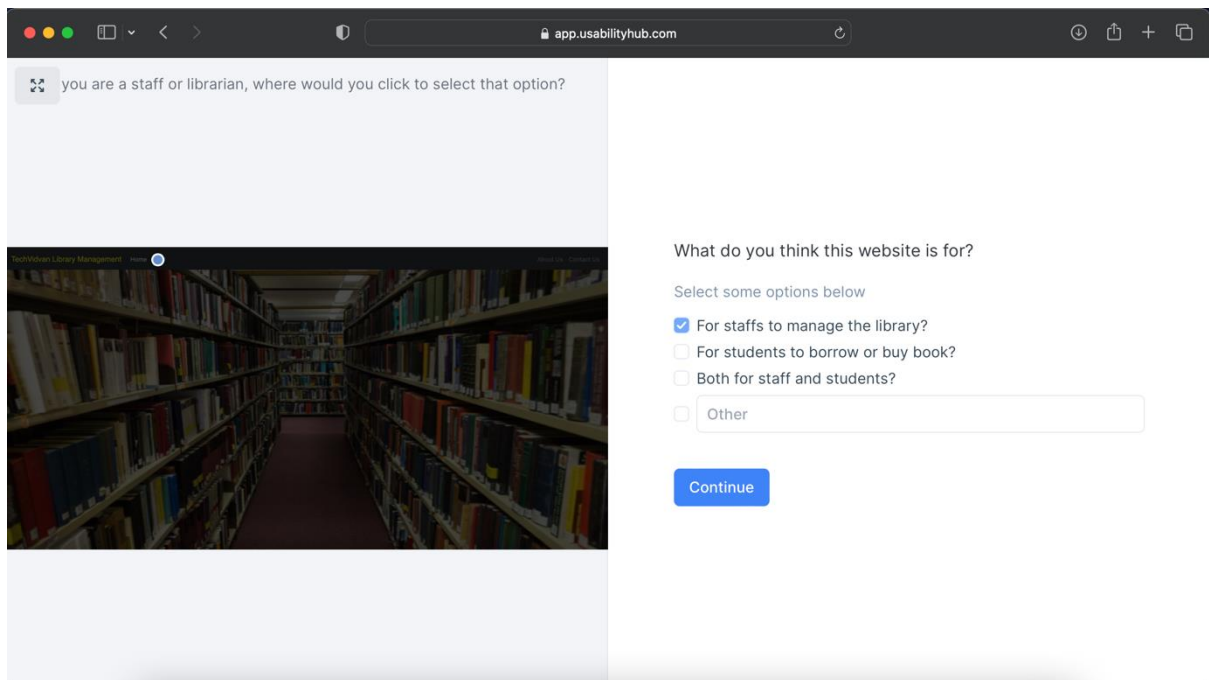
On the first page, we are asking a question on the top “If you are a staff or librarian, where would you click to select that option?”



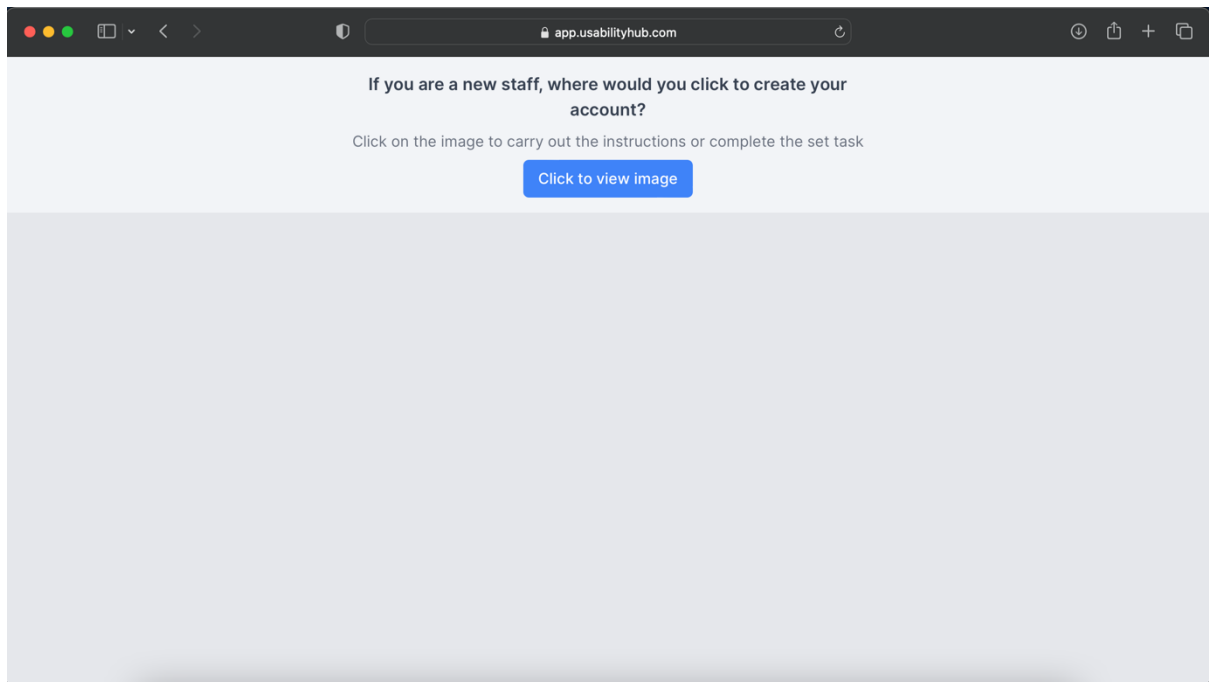
Then the users must click on the image according to the instructions.



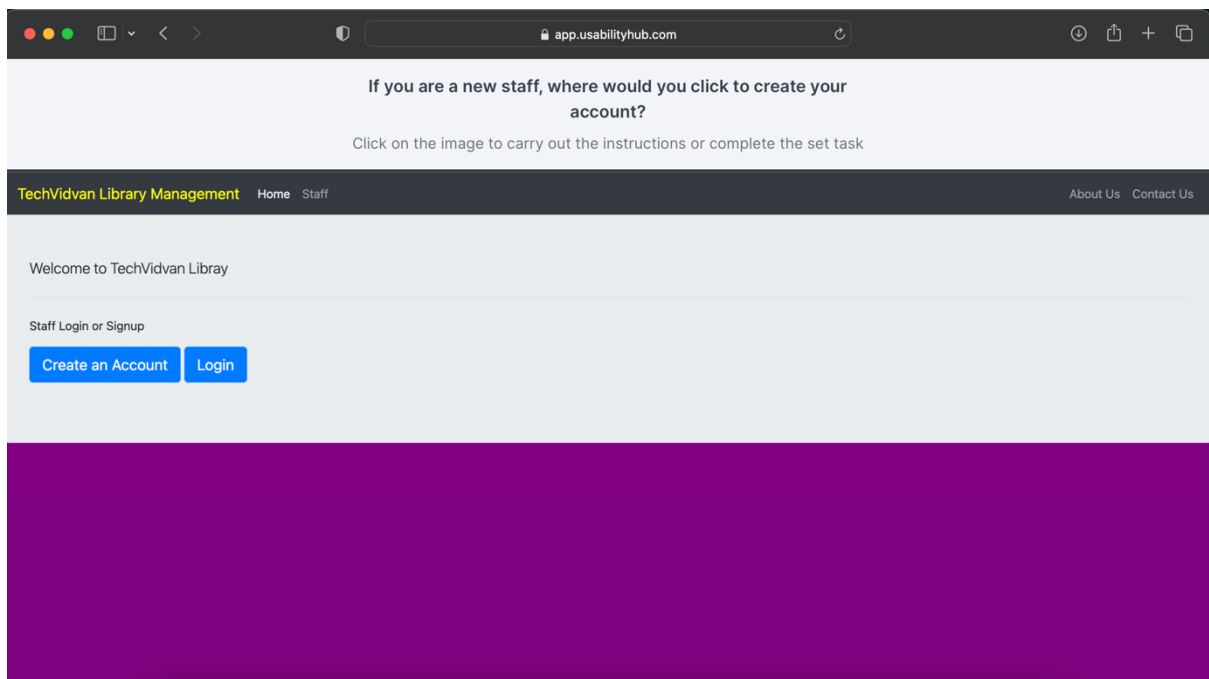
Then the user was asked questions based on the user's first impression of the software.



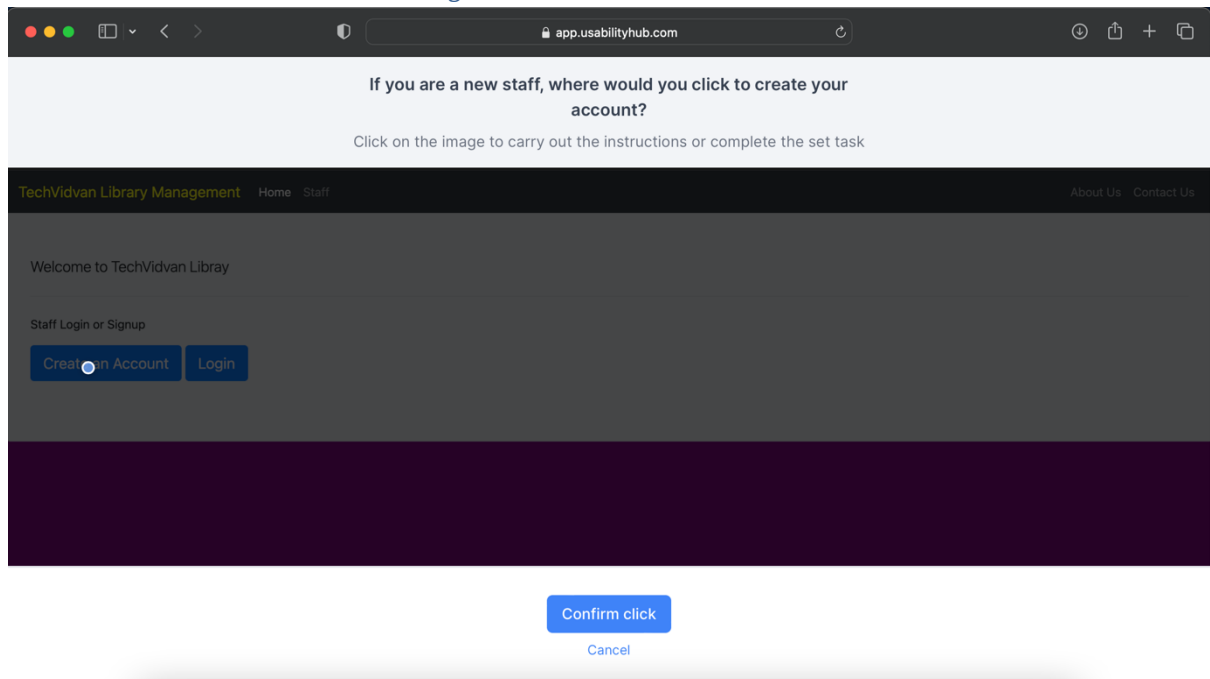
Then we created another test for our next registration/login page on the same link to ensure the user can easily find the required tools. So, we are asking where the user would click to create a new account.



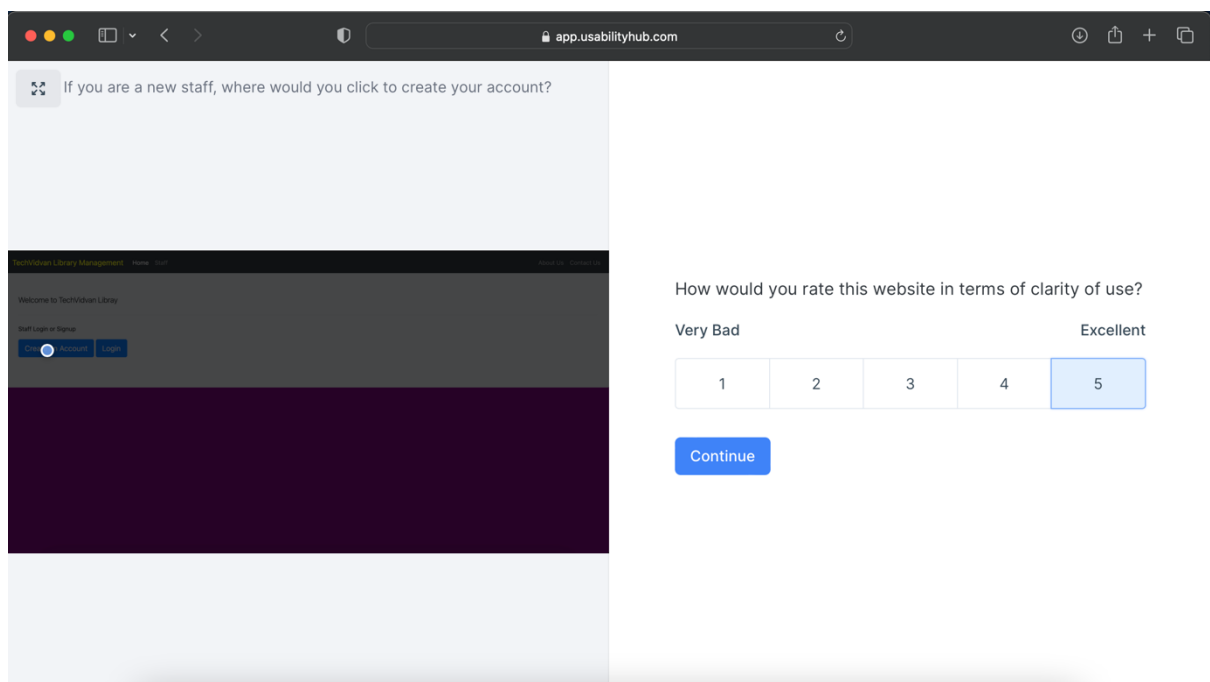
Then the image of the page is shown to the user to click on the location the user thinks is accurate by following the instructions.



Then the users must click on the image.



Then asking for feedback to rate the website in terms of clarity of use.



Results:

We requested some friends to participate in the tests, and below are the test results based on the questions we asked.

Test result link: <https://app.usabilityhub.com/tests/f57983ac9a28/results/55b5f9016b5a>

1. First click

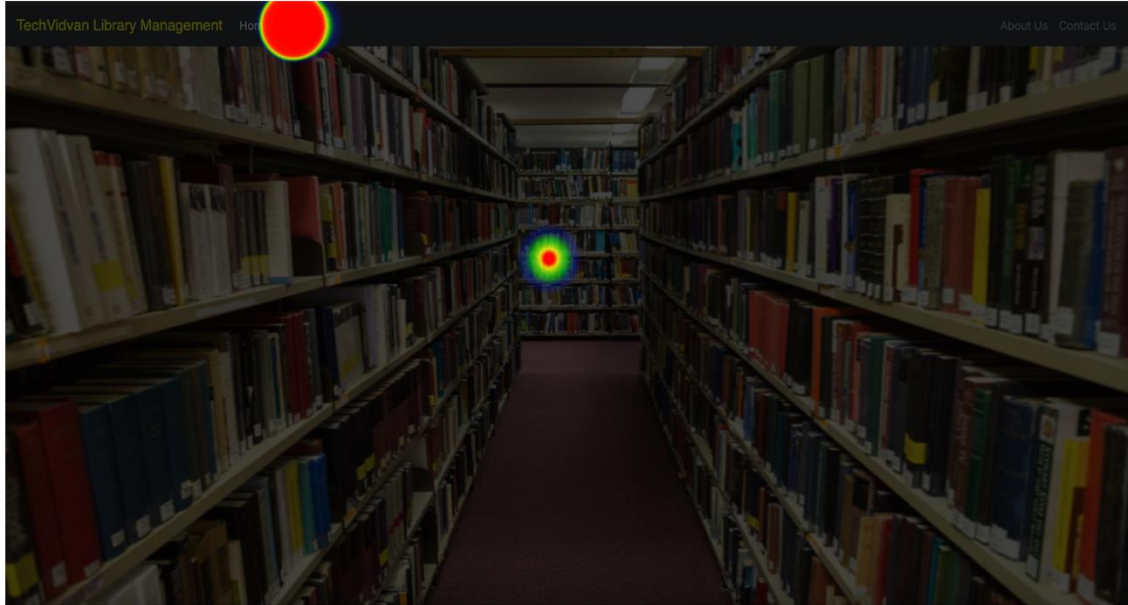
If you are a staff or librarian, where would you click to select that option?

Image

Heat map

Click map

Export heat map as image



1a. Multiple Choice question

What do you think this website is for?

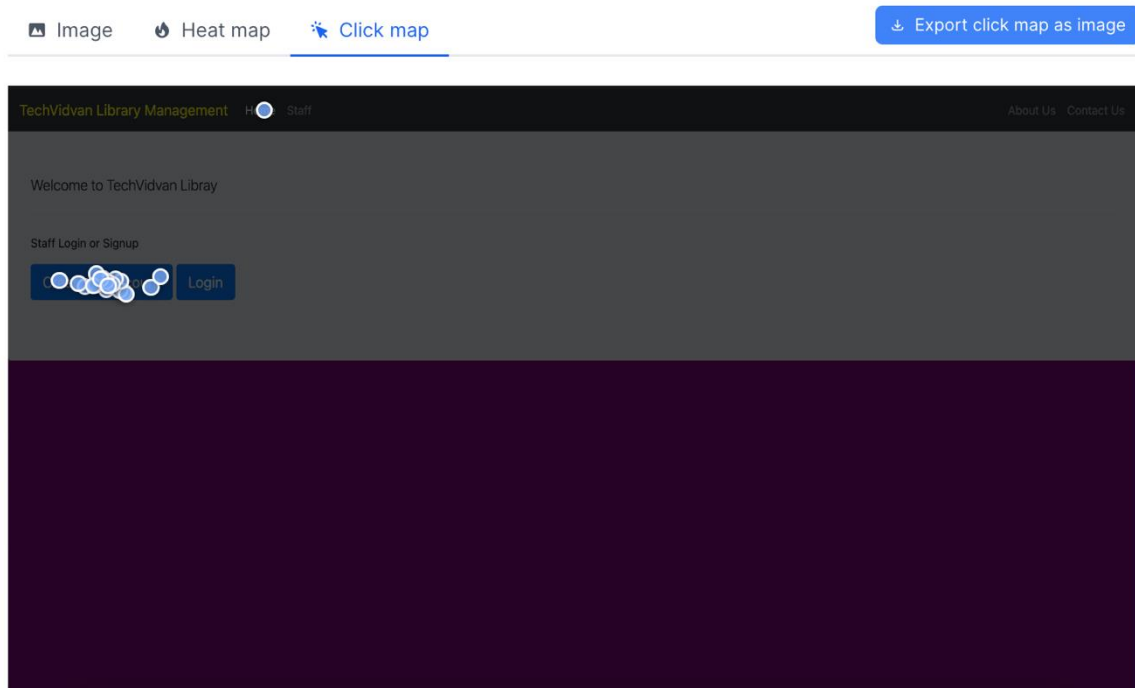
Totals

Answers 23

For staffs to manage the library?	91%	21	
Both for staff and students?	9%	2	
For students to borrow or buy book?	0%	0	
Other	0%	0	

2. First click

If you are a new staff, where would you click to create your account?



2a. Linear scale question

How would you rate this website in terms of clarity of use?

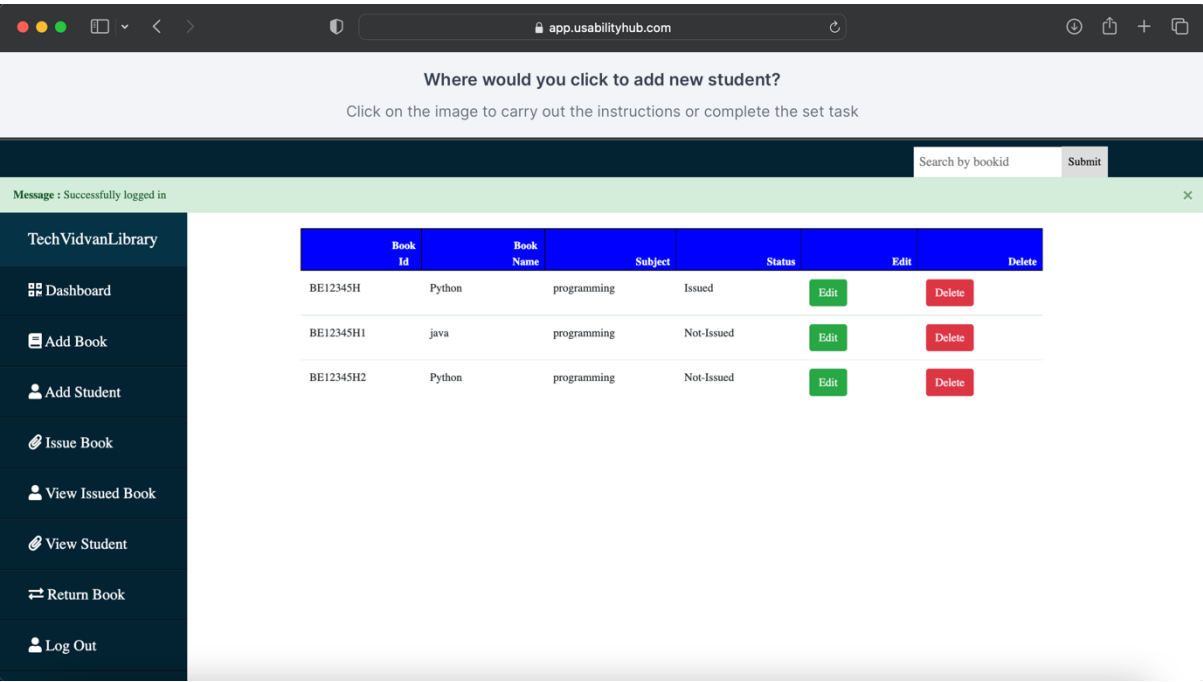
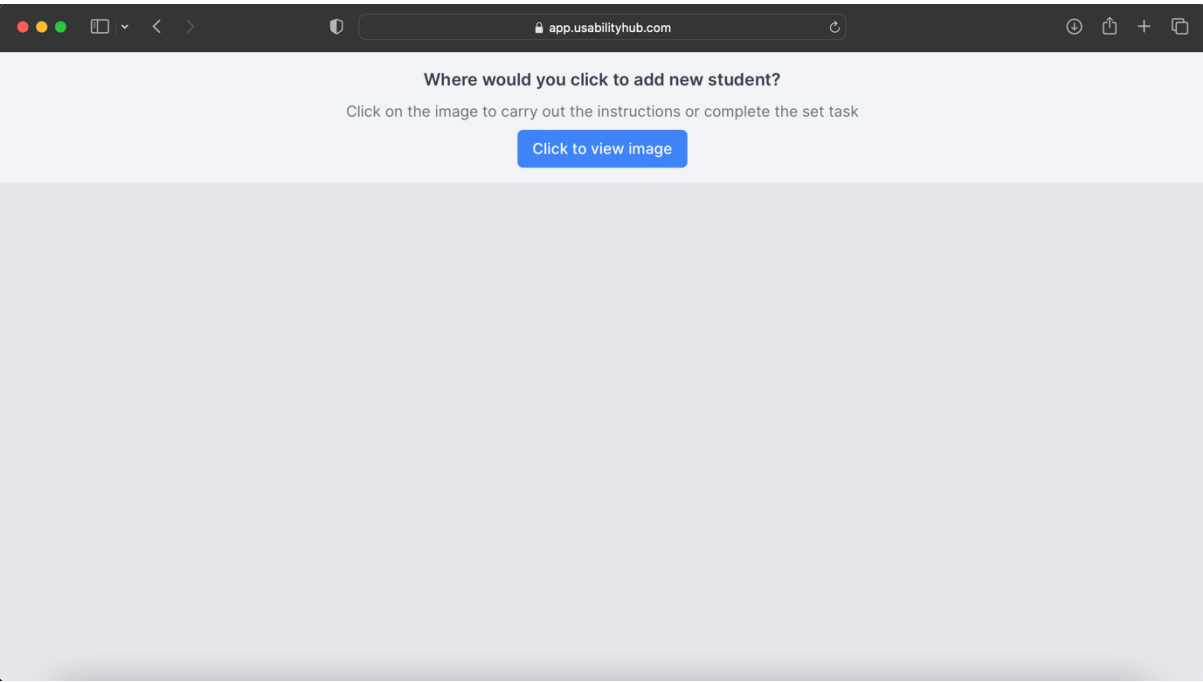


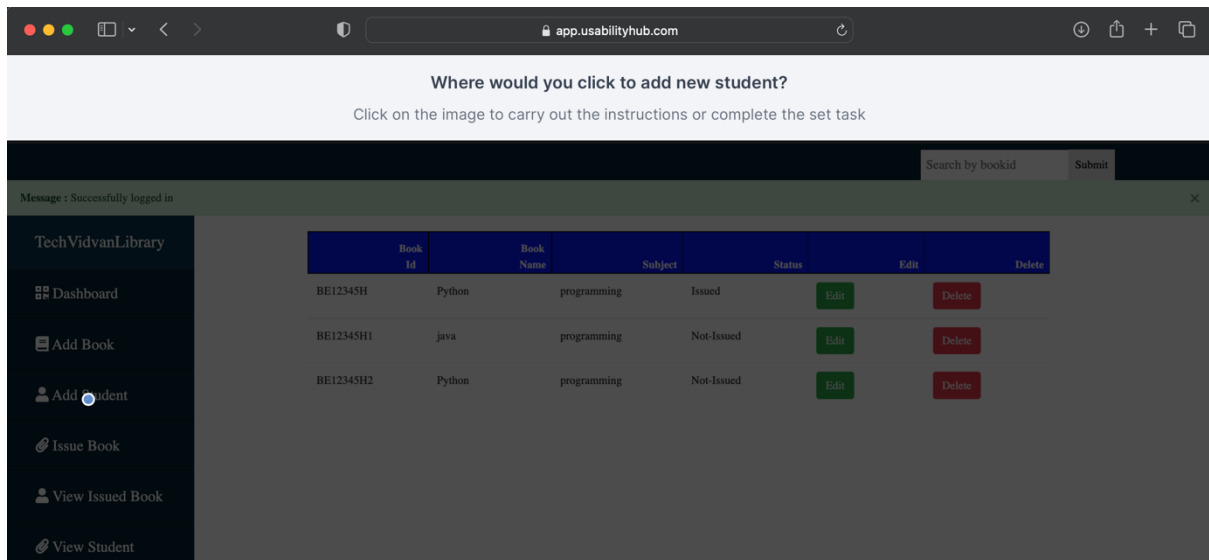
Although our sample size is not enough to decide based on the data shown in the results, most users can quickly identify the location they must navigate to finish their task. Some users answer that this website is for students and staff. However, the system is designed for the librarian staff. Therefore, we should mention that in our software.

Test 2. We created another usability test for the staff/librarian using the first-click method and generated a shareable test link that is used to get feedback from users.

Test link: <https://app.usabilityhub.com/do/6ce0ceec4cb1/8bbe>

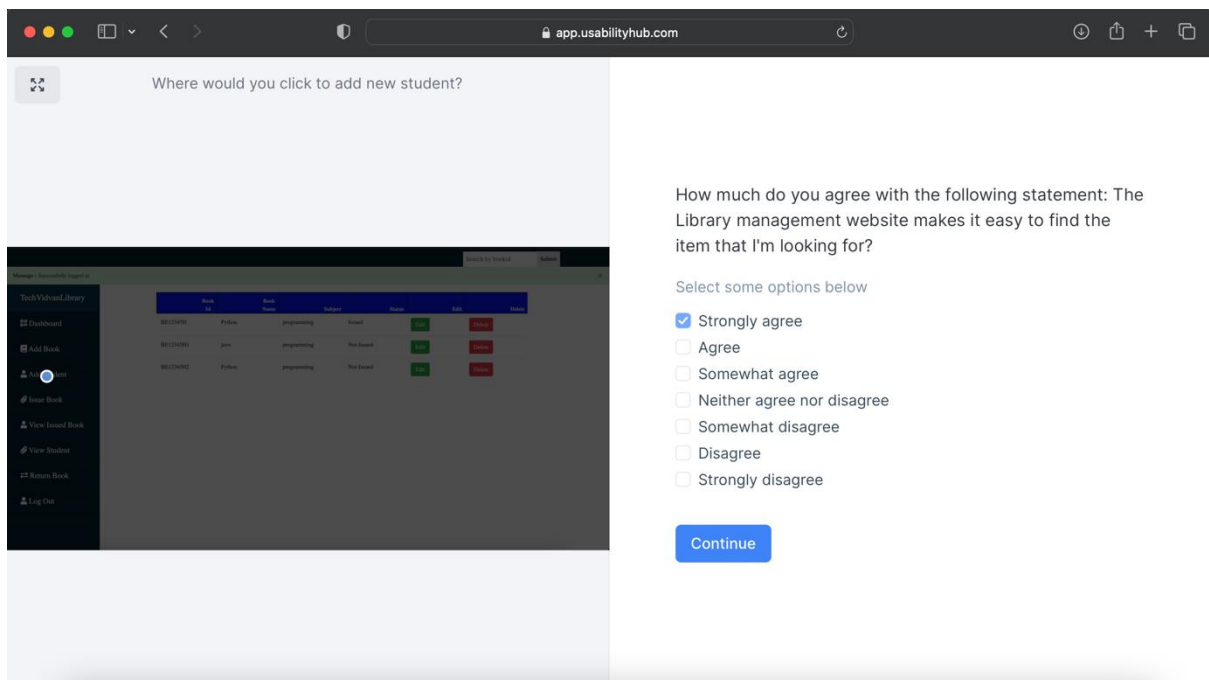
Below are the steps we followed as same as the above test:

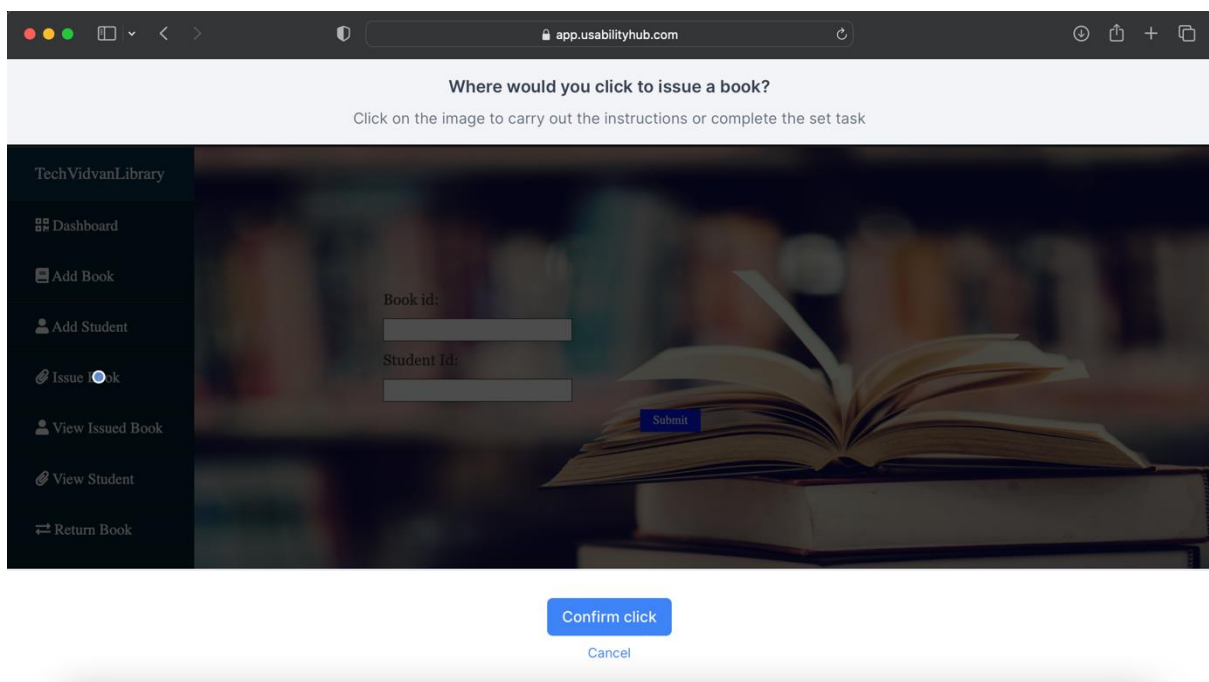
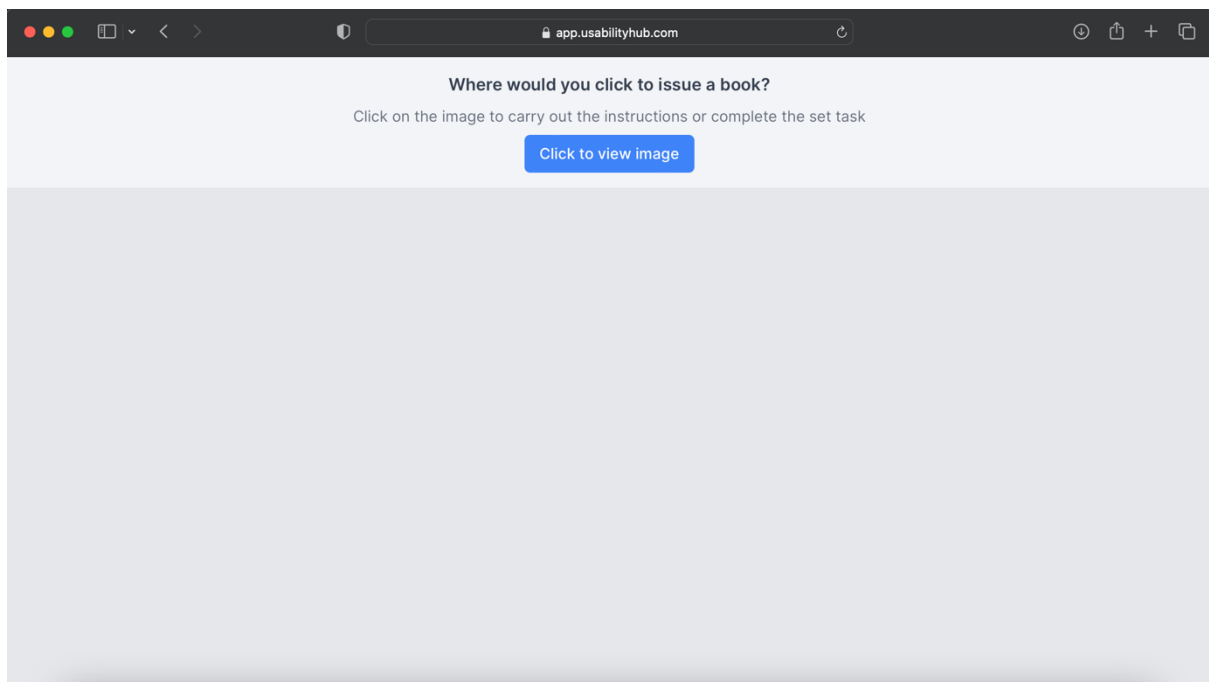


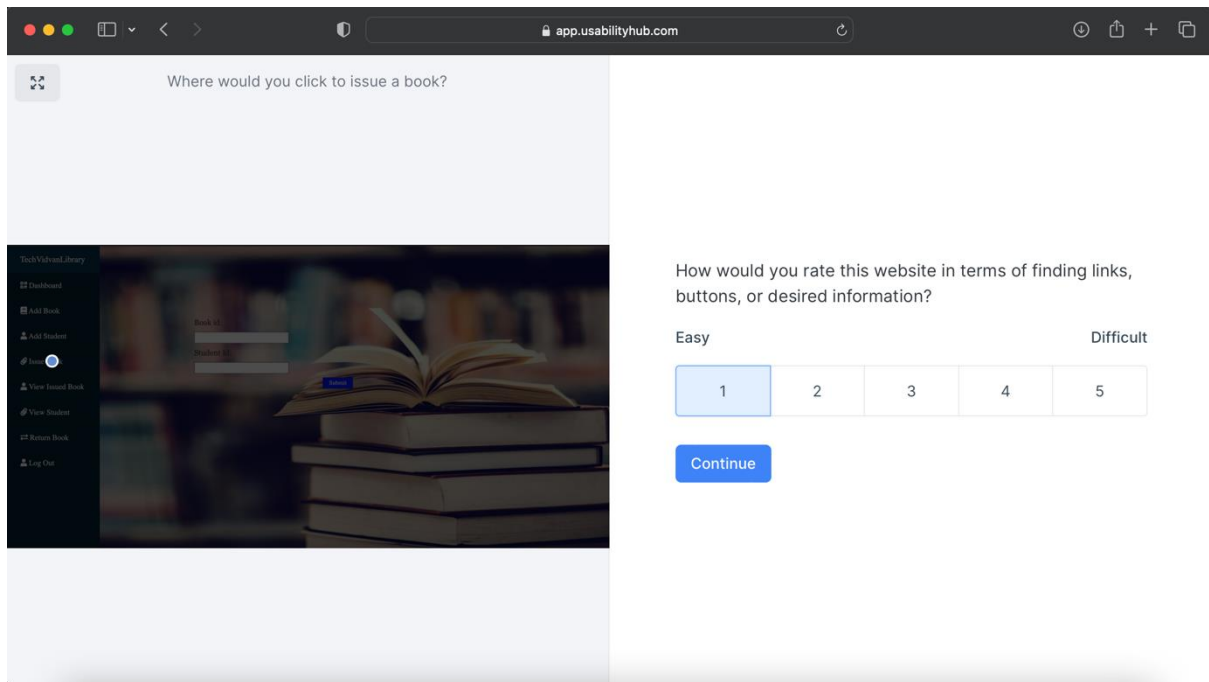


Confirm click

Cancel





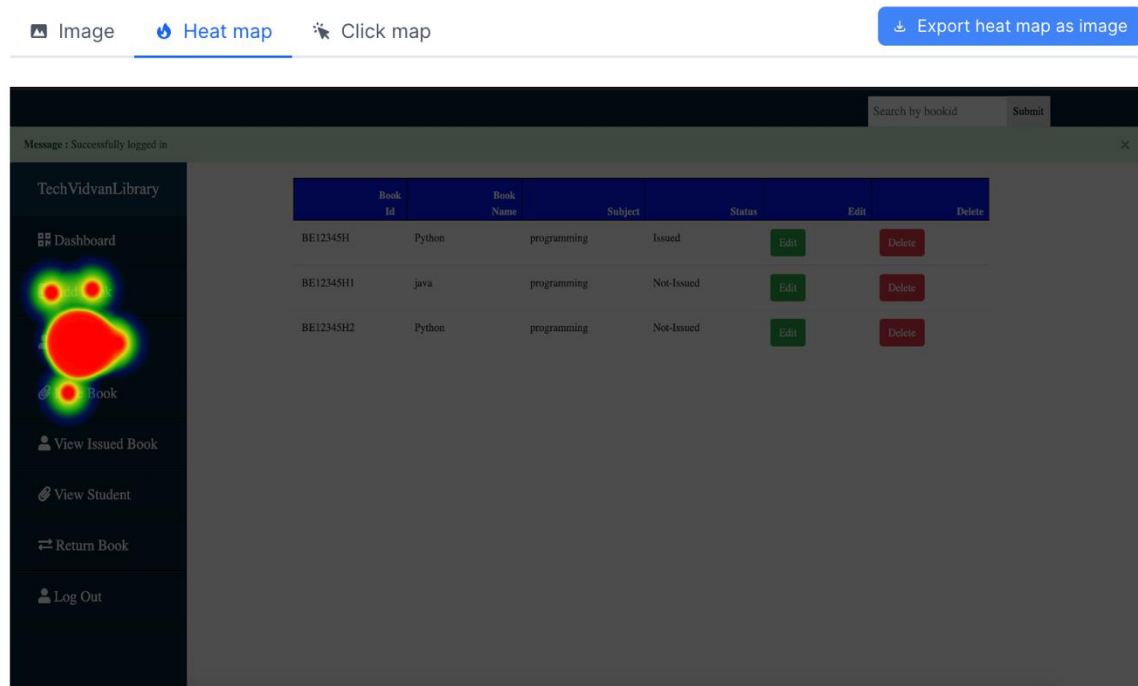


Results:

Test result link: <https://app.usabilityhub.com/tests/6ce0ceec4cb1/results/4f457d2f0ca4>

1. First click

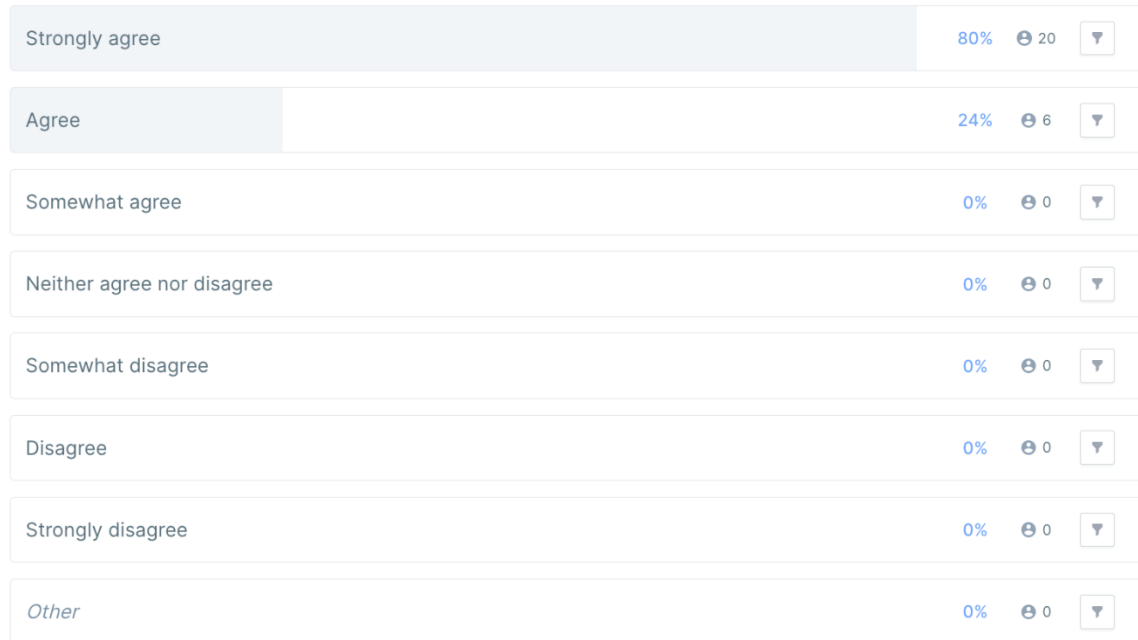
Where would you click to add new student?



1a. Multiple Choice question

How much do you agree with the following statement: The Library management website makes it easy to find the item that I'm looking for?

Totals Answers 25

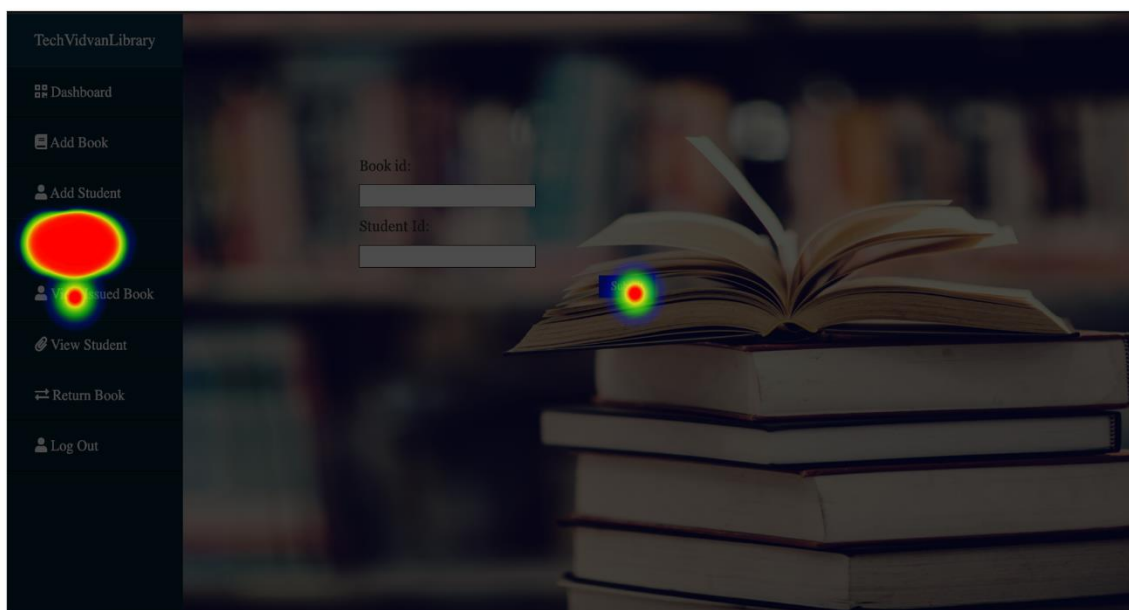


2. First click

Where would you click to issue a book?

Image Heat map Click map

Export heat map as image



2a. Linear scale question

How would you rate this website in terms of finding links, buttons, or desired information?



The design for the staff page seems reasonable, as the users were selected in the correct position. Although the sample size is not big enough to conclude the decision, overall, it looks good as it is easy to find the interface components that the user is looking for.

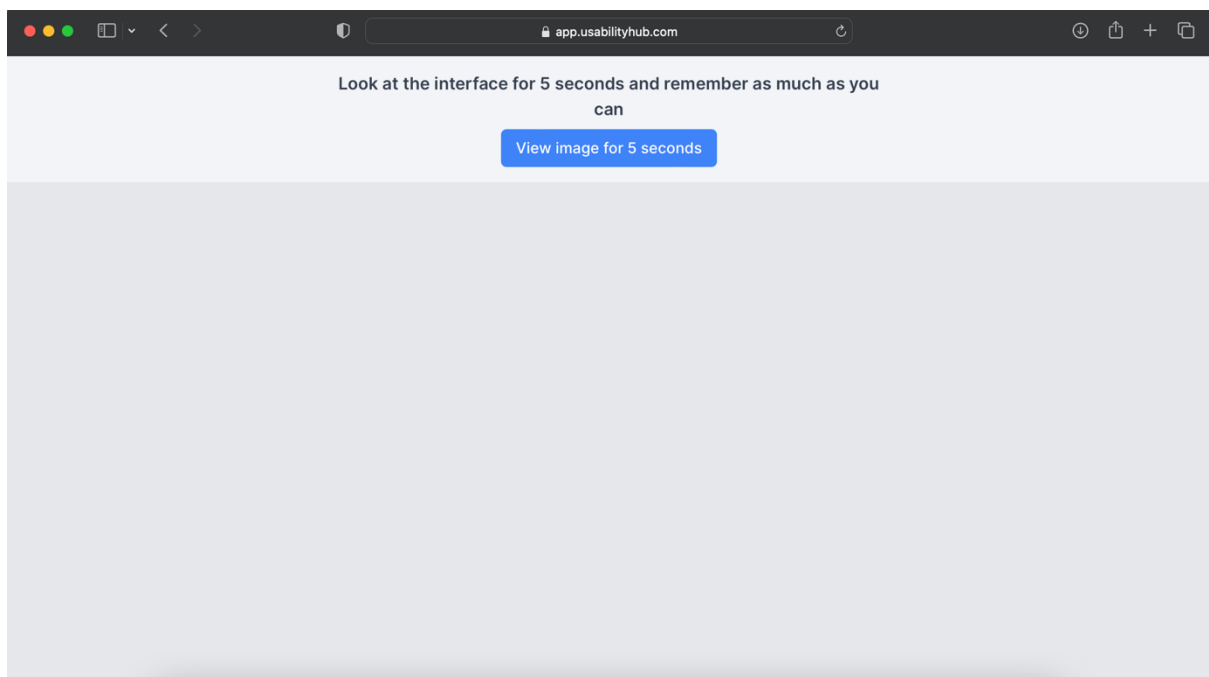
Five-second Test:

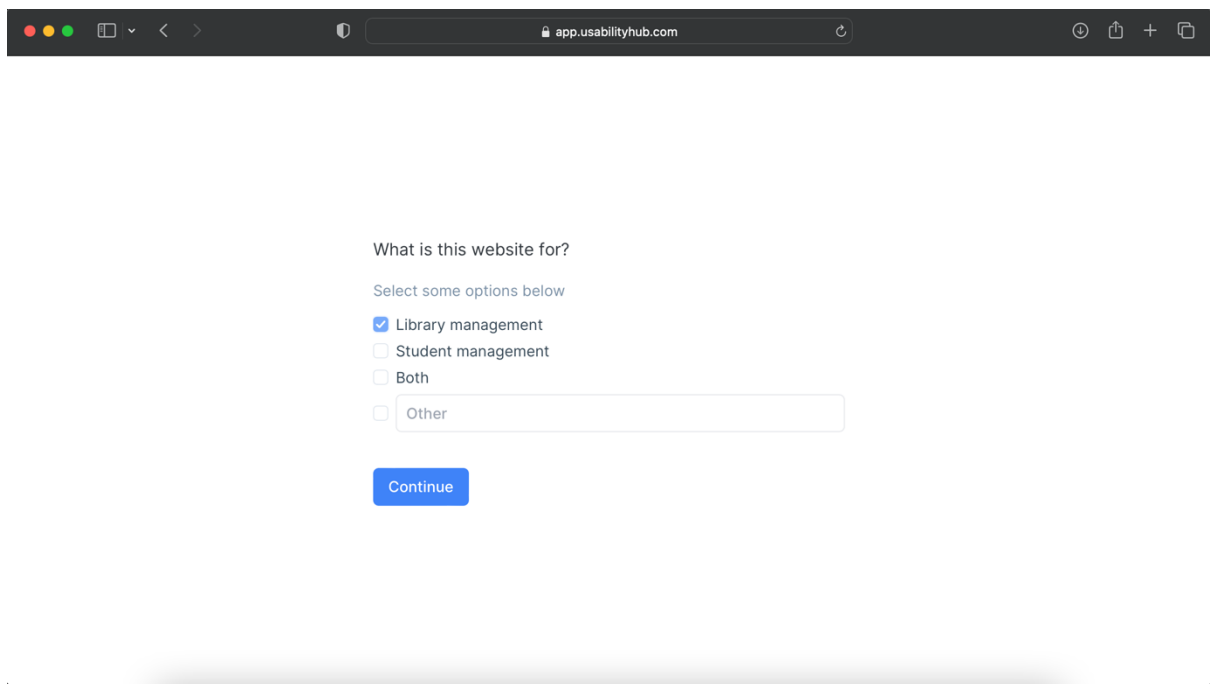
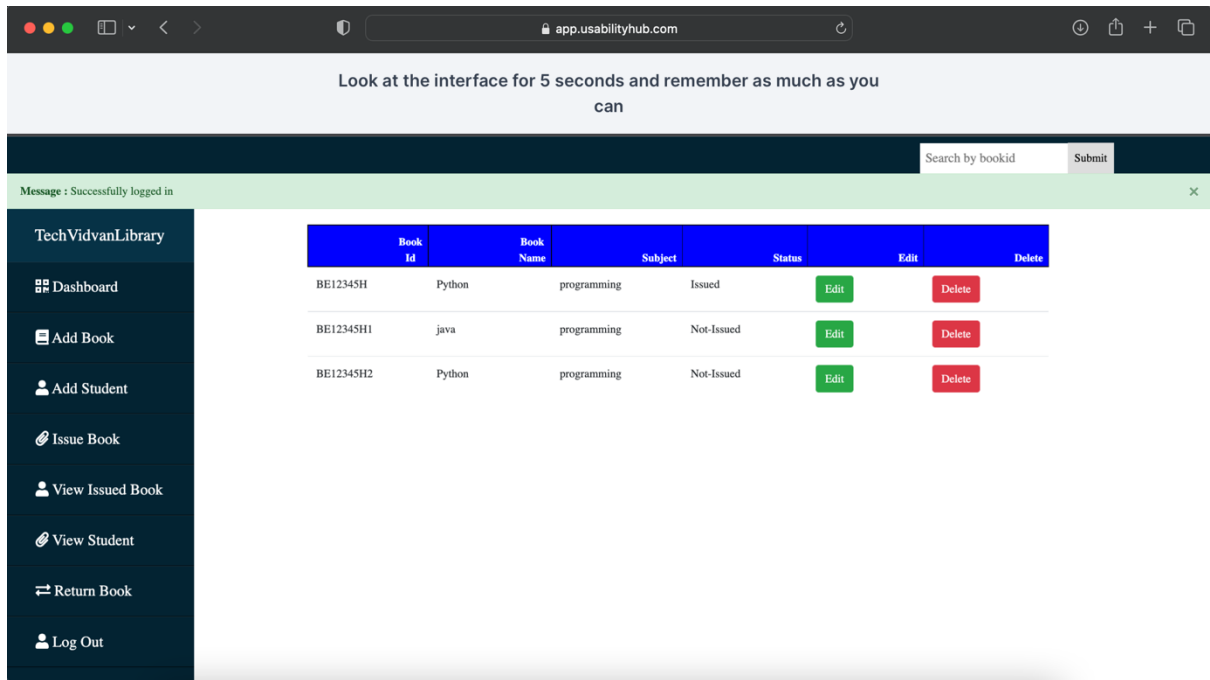
Five-second tests are a user research technique that allows us to assess the impression that users form of a design within the first five seconds of viewing it. They are frequently used to determine if web pages effectively convey the message they are meant to.

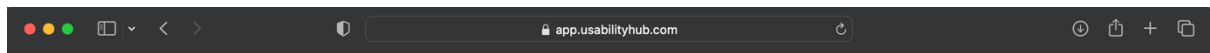
We created a 5-second test for our software in Usability Hub (<https://usabilityhub.com>) and generated a shareable test link to get user feedback.

Test link: <https://app.usabilityhub.com/do/6790c00fcc9d/d27b>

Below are the steps of the tests:





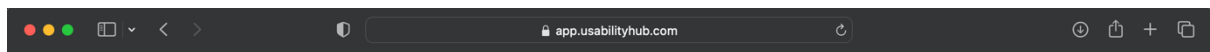


Who would find this website useful?

Select some options below

- ☒ Librarian and library staffs
- ☐ Students
- ☐ General public
- ☐ Other

Continue

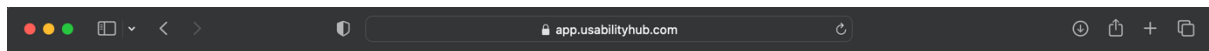


How easy it would be to navigate through this website?

Very Easy Very Difficult

1	2	3	4	5	6
---	---	---	---	---	---

Continue



How would you rate this website in terms of design?

Vary Bad

Excellent

1	2	3	4	5
---	---	---	---	---

Continue

Results:

Test result link: <https://app.usabilityhub.com/tests/6790c00fcc9d/results/7c1d2e4c343b>

1a. Multiple Choice question

What is this website for?

Totals Answers 19

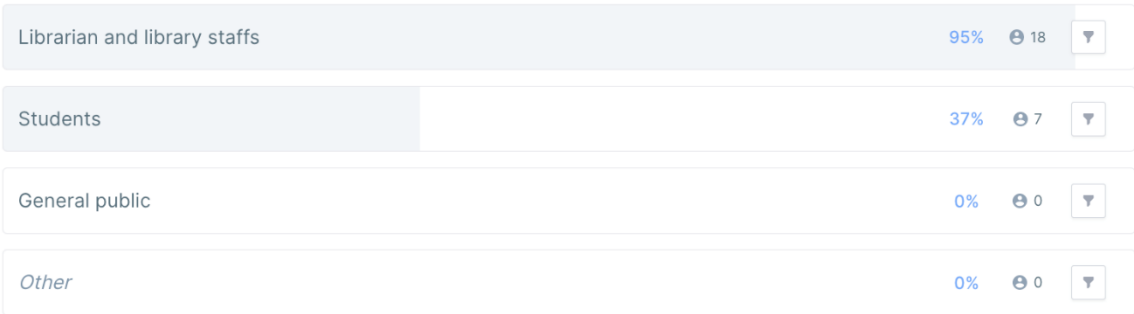
Library management	84%	16	
Both	11%	2	
Student management	5%	1	
Other	0%	0	

Some test takers answered that this software is for both library and student management, but it is not, so we need to fix that to convey the exact message that it is only for library management. As the sample size is not large enough to decide, we need to hire more test takers to perform the test to make an informed decision.

1b. Multiple Choice question

Who would find this website useful?

Totals Answers 19



Most of the test takers get the correct idea of what this application is about.

1c. Linear scale question

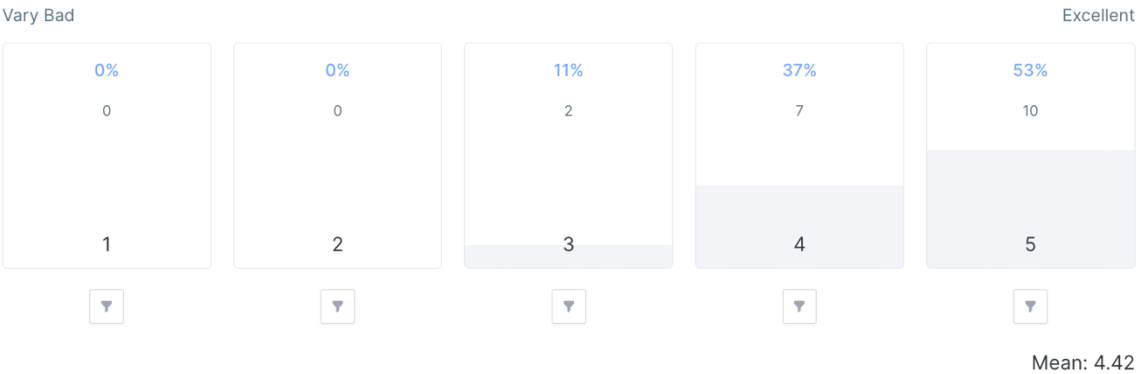
How easy it would be to navigate through this website?



Some users find the website difficult to navigate, so we must investigate and fix that issue in our software.

1d. Linear scale question

How would you rate this website in terms of design?



Most of the test takers rated the software from 3 to 5 out of five in terms of design which is not bad. Still, there is a scope to recheck the design and fix if there are any issues, or we could take another test where we could ask the test takers what things or components they think can be improved.

Performance Test:

Performance testing is a non-functional software testing technique that assesses how well an application performs in speed, scalability, responsiveness, and stability under expected workloads. Finding bugs or flaws in software is not the point of this. Various performance testing methodologies measure outcomes by standards and benchmarks. Developers can identify bottlenecks and remove them with the help of performance testing.

Types of performance testing for software:

Load testing:

Load testing assesses how well a system performs as the workload grows. That workload could indicate concurrent users or transactions. The system is evaluated to measure response time and system endurance as the workload increases.

Stress testing:

Stress testing is typically done to determine the system's maximum capacity. This test assesses the system's robustness under extreme load and aids application administrators in deciding if the system will function well if the current load is significantly higher than the anticipated maximum.

Scalability testing:

The software's ability to handle increasing workloads is evaluated through scalability testing. This can be determined by continually improving the user load or data volume while monitoring system performance. Additionally, even though resources like CPUs and memory are revised, the workload could remain unchanged.

Breakpoint testing:

Stress testing and breakpoint testing are related. Over time, a gradually increasing load is supplied while the system is watched for certain failure circumstances.

Endurance testing:

Ensuring the program can endure the anticipated load over an extended period.

Spike testing:

It evaluates how the system responds to unexpectedly high user load spikes.

Different tools available to conduct performance testing:

Web LOAD:

A web application load and enterprise standard performance testing tool. Web LOAD is the tool of choice for businesses with a high user load and complicated testing requirements. Creating load from the cloud and on-premises servers enables us to do load and stress testing on any internet application.

Load Ninja:

We can quickly construct scripless advanced load tests using Load Ninja by Smart Bear, which also decreases testing time in half, replaces load emulators with actual browsers, and obtains actionable browser-based metrics.

Load View:

The load and stress testing process is hassle-free with the help of Load View, a fully managed, on-demand load testing tool.

Apache JMeter:

It is a Java platform program that is an open-source load-testing tool. It can be incorporated with the test plan and is thought of as a performance testing tool. You can make a functional test plan in addition to the load test plan. This tool can be installed into a server or network to monitor performance and examine how it operates in various scenarios. It was initially designed to test web applications, but over time, its applicability expanded.

Micro Focus LoadRunner:

It is possible to utilize this Micro Focus product as a Performance Testing tool. Its Micro Focus software department offers this as a Micro Focus product. Additionally, it is helpful in figuring out how the system will operate and perform under actual load.

Rational Performance Tester:

Rational performance tester is an automated performance testing tool that can be used for a web application or a server-based application where input and output are involved.

Neo Load:

For enterprise organizations that regularly test applications and APIs, Neo Load is the most automated performance testing platform.

Loadster:

Loadster is a desktop-based advanced HTTP (Hypertext Transfer Protocol) load testing tool. The web browser can record scripts that are easy to use and record.

Page Speed:

Page Speed Insights (PSI) reports a page's performance on desktop and mobile devices and offers recommendations for improving that page.

Python timeit:

Python's timeit function can measure how well our code works. It is a method in the Python library to calculate how long the provided code snippet will take to execute.

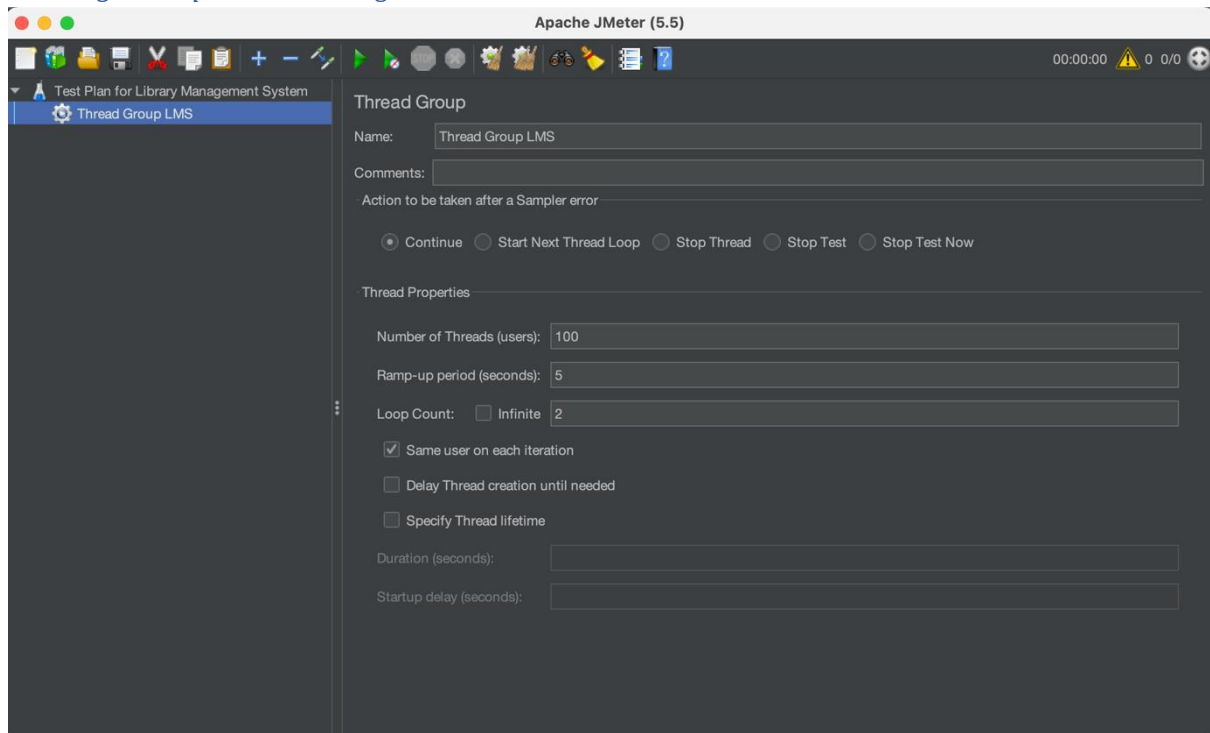
There are many other tools available to perform performance and load tests.

As our software is locally hosted on a personal computer, we will perform performance and load testing using **Apache JMeter**. Apache JMeter is a popular open-source performance testing tool that can be used to conduct performance tests on servers/systems and web applications.

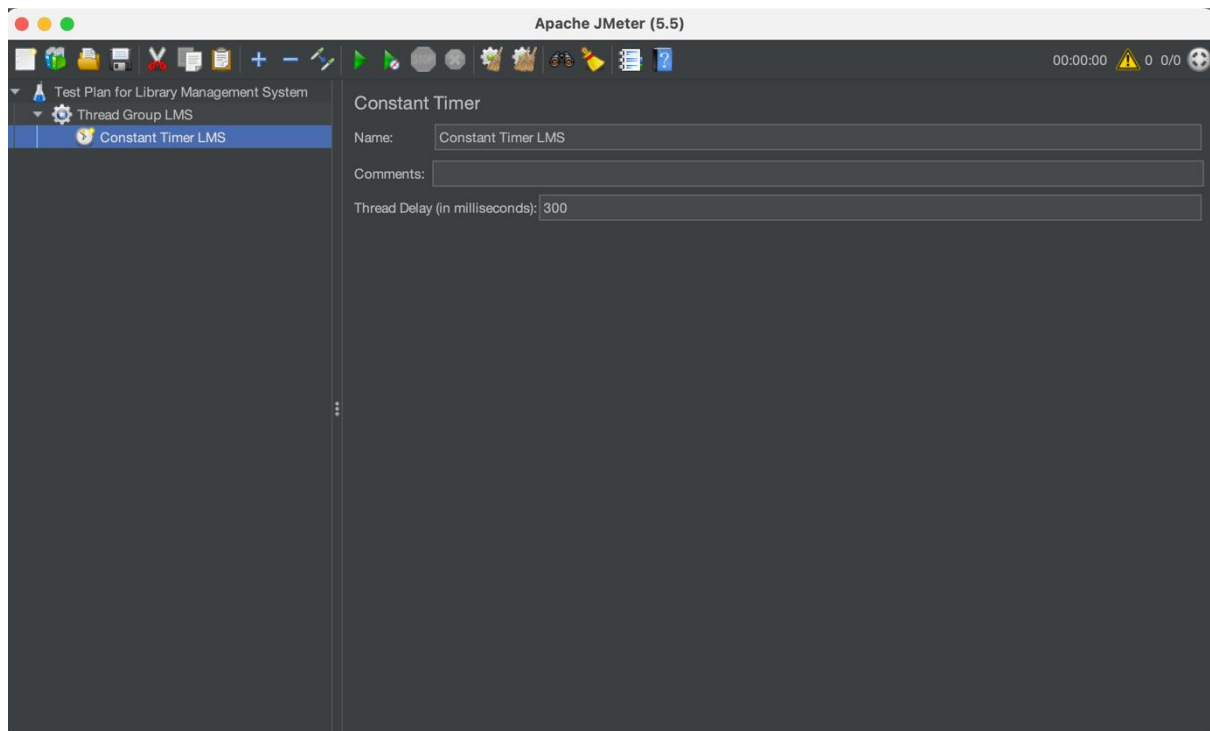
JMeter helps load tests on web servers, websites, and web applications by simulating real-user behaviours and testing environments. JMeter provides a user-friendly GUI, easy installation, testing strategies, simulation, and many other performance testing features.

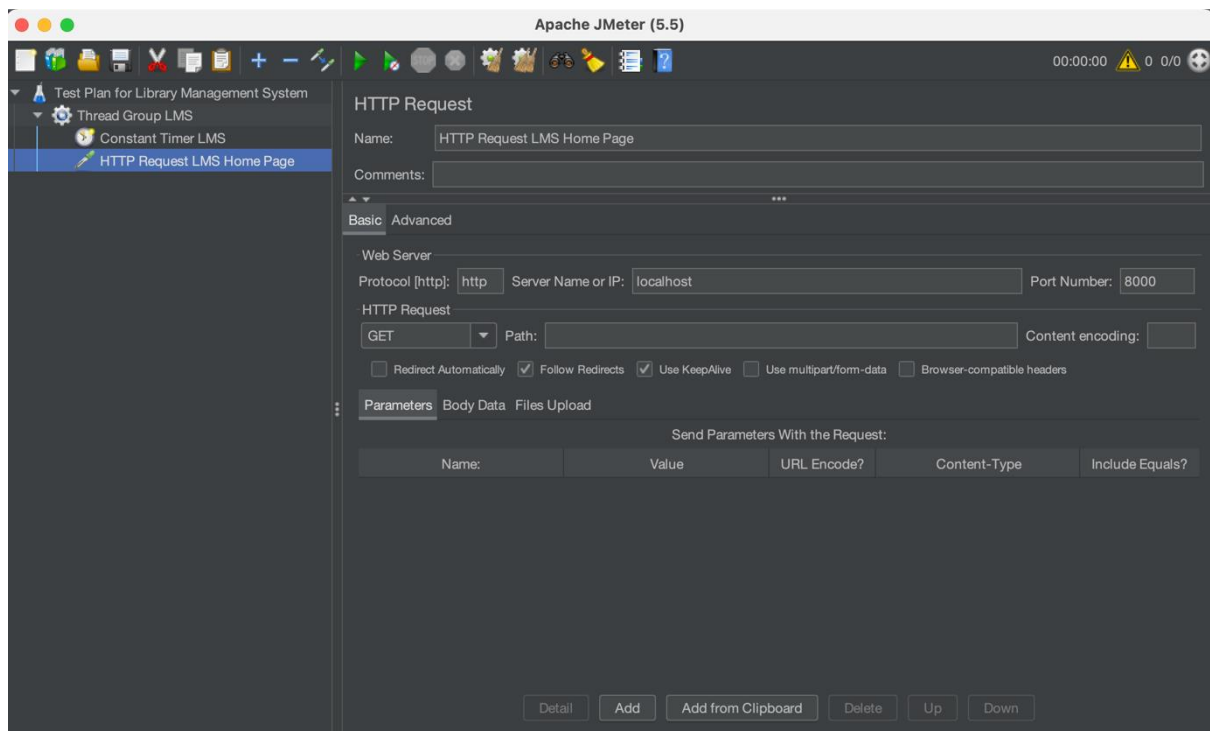
Creating performance test using JMeter:

Creating a test plan and adding several threads

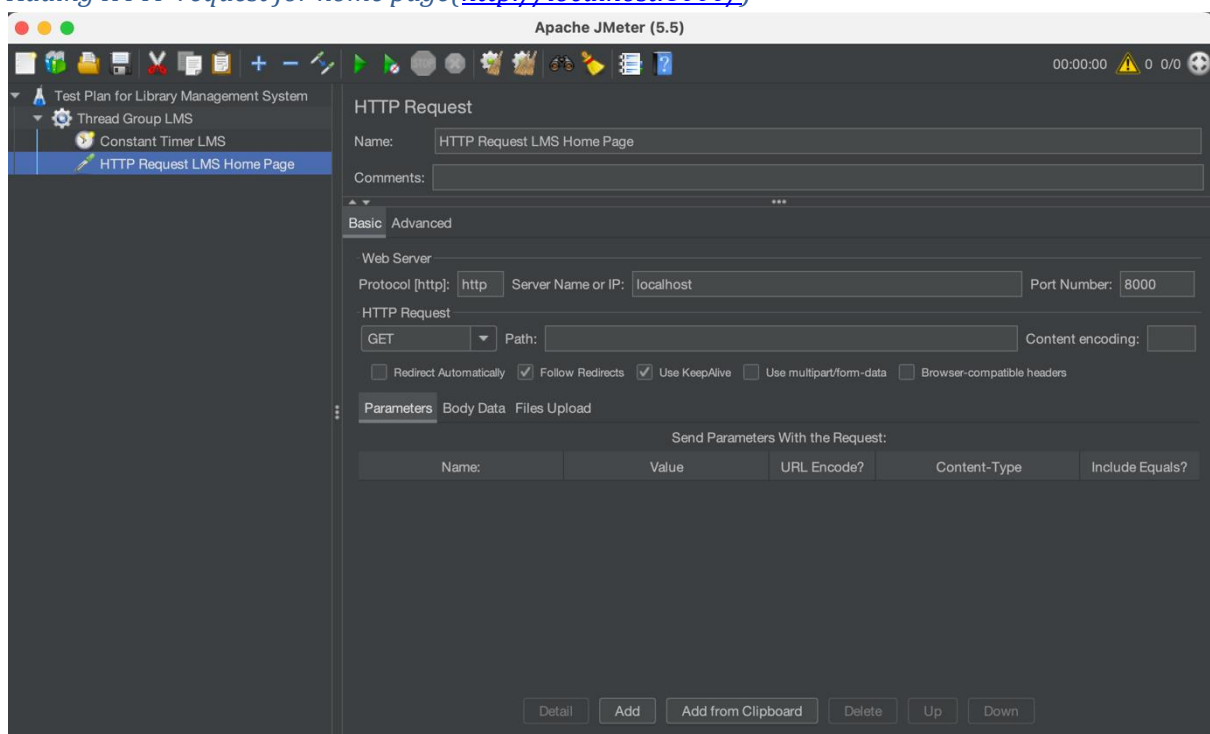


Adding timer

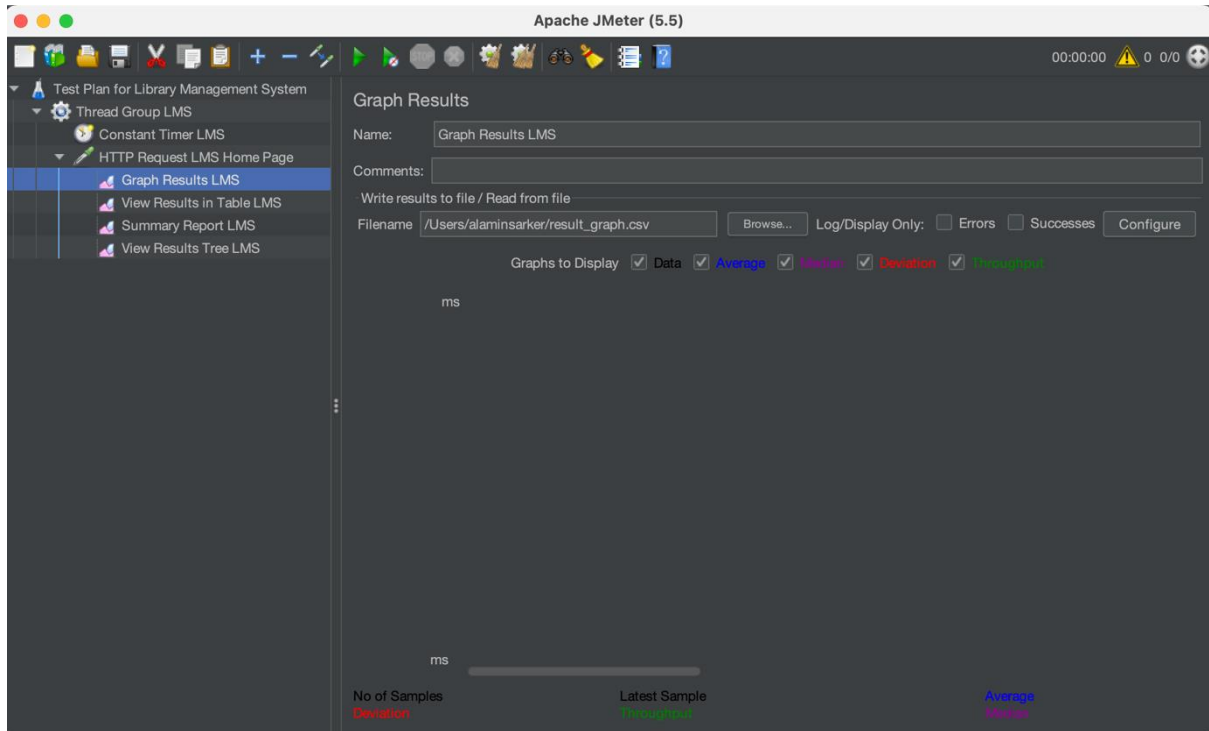




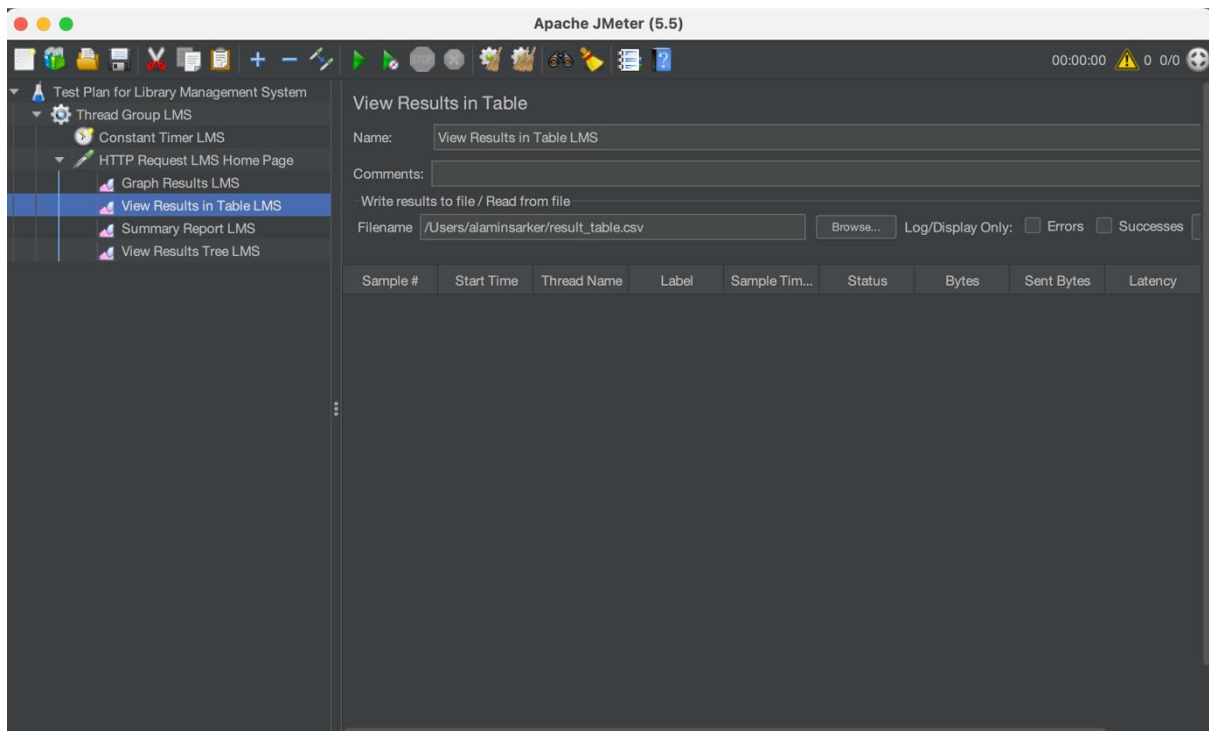
Adding HTTP request for home page(<http://localhost:8000/>)



Adding graph listener to see output or result as a graph.



Adding another listener to see results in the table view.



Adding a listener for summary reports.

The screenshot shows the Apache JMeter (5.5) interface. On the left, the 'Test Plan for Library Management System' tree is expanded, showing a 'Thread Group LMS' with several listeners. The 'Summary Report LMS' listener is selected and highlighted in blue. The main panel displays the configuration for the 'Summary Report' listener. The 'Name' field is set to 'Summary Report LMS'. The 'Comments' field is empty. The 'Write results to file / Read from file' section shows the 'Filename' as '/Users/alaminsarker/result_summary.csv' with a 'Browse...' button. The 'Log/Display Only' section has checkboxes for 'Errors' and 'Successes', both of which are unchecked. A 'Configure' button is present. Below this, a table displays test results. The table has columns: Label, # Samples, Average, Min, Max, Std. Dev., Error %, Through..., Received..., Sent KB/..., and Avg. Bytes. The first row is labeled 'TOTAL' and shows values: # Samples: 0, Average: 0, Min: #N/A, Max: #N/A, Std. Dev.: 0.00, Error %: 0.00%, Through...: .0/hour, Received...: 0.00, Sent KB/...: 0.00, Avg. Bytes: .0. At the bottom, there are checkboxes for 'Include group name in label?' (unchecked), 'Save Table Data' (button), and 'Save Table Header' (checked).

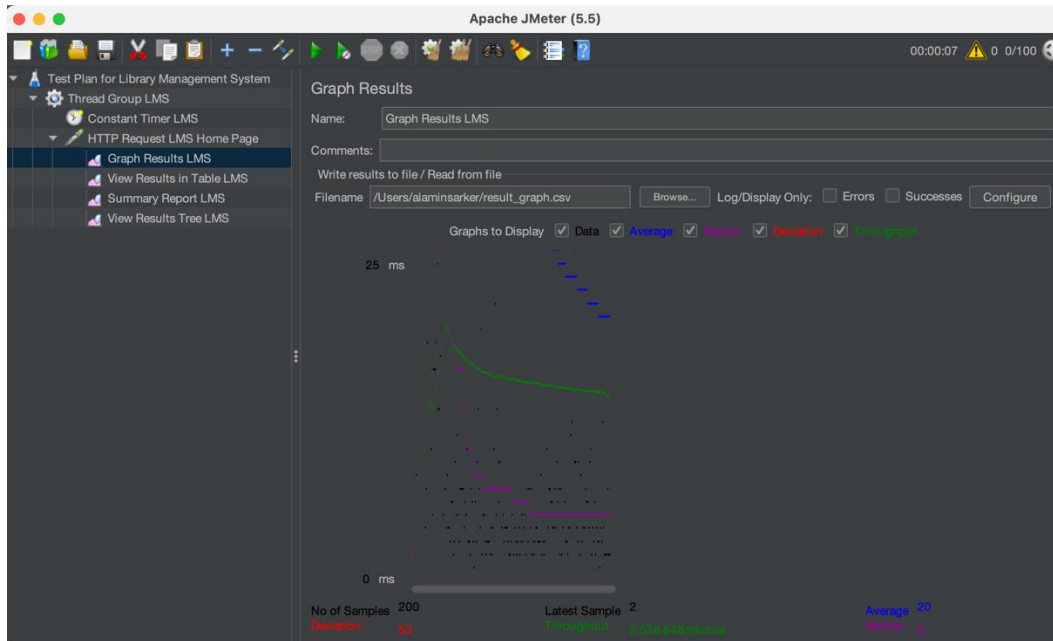
Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Through...	Received...	Sent KB/...	Avg. Bytes
TOTAL	0	0	#N/A	#N/A	0.00	0.00%	.0/hour	0.00	0.00	.0

Adding a listener to view the results in a tree.

The screenshot shows the Apache JMeter (5.5) interface. On the left, the 'Test Plan for Library Management System' tree is expanded, showing a 'Thread Group LMS' with several listeners. The 'View Results Tree LMS' listener is selected and highlighted in blue. The main panel displays the configuration for the 'View Results Tree' listener. The 'Name' field is set to 'View Results Tree LMS'. The 'Comments' field is empty. The 'Write results to file / Read from file' section shows the 'Filename' as '/Users/alaminsarker/result_tree.csv' with a 'Browse...' button. The 'Log/Display Only' section has checkboxes for 'Errors' and 'Successes', both of which are unchecked. A 'Configure' button is present. Below this, there is a search bar with a 'Search' button and a 'Reset' button. The 'Text' field is set to 'Sampler result'. At the bottom, there is a checkbox for 'Scroll automatically?' (unchecked) and tabs for 'Raw' and 'Parsed'.

Running the test:

Graph result.



Results in the table:

Apache JMeter (5.5)

Test Plan for Library Management System

- Thread Group LMS
 - Constant Timer LMS
 - HTTP Request LMS Home Page
 - Graph Results LMS
 - View Results in Table LMS**
 - Summary Report LMS
 - View Results Tree LMS

View Results in Table

Name: View Results in Table LMS

Comments:

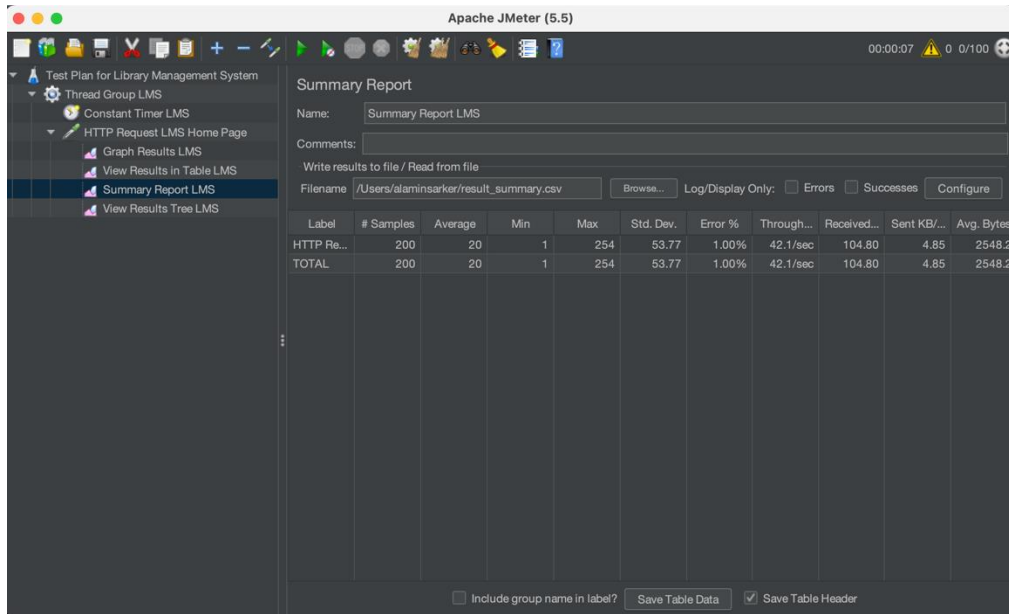
Write results to file / Read from file

Filename: /Users/alaminsarker/result_table.csv

Log/Display Only: ☐ Errors ☐ Successes

Sample #	Start Time	Thread Name	Label	Sample Tim...	Status	Bytes	Sent Bytes	Latency
1	12:03:25.689	Thread Gro...	HTTP Requ...	117	✖	2360	0	0
2	12:03:25.689	Thread Gro...	HTTP Requ...	122	✖	2374	0	0
3	12:03:25.692	Thread Gro...	HTTP Requ...	248	✔	2550	119	233
4	12:03:25.932	Thread Gro...	HTTP Requ...	8	✔	2550	119	7
5	12:03:25.692	Thread Gro...	HTTP Requ...	248	✔	2550	119	235
6	12:03:25.692	Thread Gro...	HTTP Requ...	248	✔	2550	119	221
7	12:03:25.692	Thread Gro...	HTTP Requ...	248	✔	2550	119	233
8	12:03:25.692	Thread Gro...	HTTP Requ...	248	✔	2550	119	221
9	12:03:25.692	Thread Gro...	HTTP Requ...	248	✔	2550	119	235
10	12:03:25.713	Thread Gro...	HTTP Requ...	228	✔	2550	119	214
11	12:03:25.692	Thread Gro...	HTTP Requ...	249	✔	2550	119	235
12	12:03:25.886	Thread Gro...	HTTP Requ...	54	✔	2550	119	33
13	12:03:25.687	Thread Gro...	HTTP Requ...	253	✔	2550	119	230
14	12:03:25.686	Thread Gro...	HTTP Requ...	254	✔	2550	119	239
15	12:03:25.987	Thread Gro...	HTTP Requ...	4	✔	2550	119	4
16	12:03:26.030	Thread Gro...	HTTP Requ...	7	✔	2550	119	7
17	12:03:26.084	Thread Gro...	HTTP Requ...	18	✔	2550	119	18
18	12:03:26.129	Thread Gro...	HTTP Requ...	27	✔	2550	119	27
19	12:03:26.136	Thread Gro...	HTTP Requ...	27	✔	2550	119	27

Summary report:

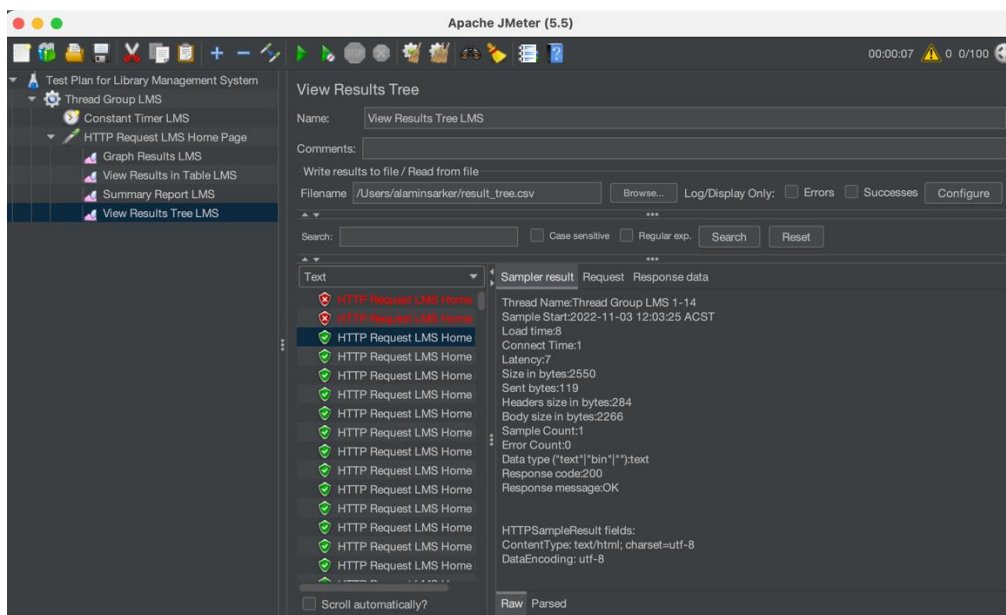


The screenshot shows the Apache JMeter (5.5) Summary Report window. The left sidebar shows a test plan for 'Library Management System' with a thread group 'LMS' containing a 'Constant Timer LMS' and an 'HTTP Request LMS Home Page'. The main area displays the 'Summary Report' for 'Summary Report LMS'. It includes fields for Name, Comments, and a 'Write results to file / Read from file' section with a 'Filename' field set to '/Users/alaminsarker/result_summary.csv'. Below this is a table with the following data:

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Through...	Received...	Sent KB/...	Avg. Bytes
HTTP Re...	200	20	1	254	53.77	1.00%	42.1/sec	104.80	4.85	2548.2
TOTAL	200	20	1	254	53.77	1.00%	42.1/sec	104.80	4.85	2548.2

At the bottom, there are checkboxes for 'Include group name in label?' and 'Save Table Header' (checked), and a 'Save Table Data' button.

View results tree.

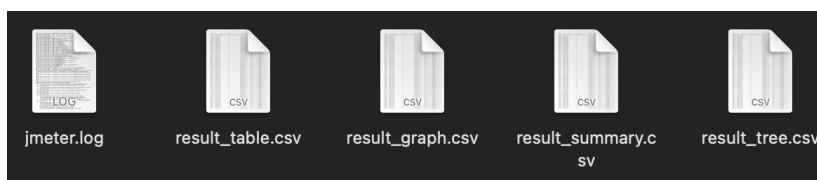


The screenshot shows the Apache JMeter (5.5) View Results Tree window. The left sidebar shows the same test plan as the previous screenshot. The main area displays the 'View Results Tree' for 'View Results Tree LMS'. It includes fields for Name, Comments, and a 'Write results to file / Read from file' section with a 'Filename' field set to '/Users/alaminsarker/result_tree.csv'. Below this is a search bar and a list of results. The first result is an 'HTTP Request LMS Home' with a status of 'Success'. The details for this result are shown on the right:

Sampler result	Request	Response data
Thread Name:Thread Group LMS 1-14 Sample Start:2022-11-03 12:03:25 ACST Load time:8 Connect Time:1 Latency:7 Size in bytes:2550 Sent bytes:119 Headers size in bytes:284 Body size in bytes:2266 Sample Count:1 Error Count:0 Data type ("text" "bin" ""):text Response code:200 Response message:OK		

At the bottom, there are checkboxes for 'Scroll automatically?' and 'Raw Parsed'.

Also, reports are generated in CSV format and automatically saved on the preferred location of the computer.



Let us run our tests using a different number of threads. So, we are increasing the number of threads to see how our application performs on loads. Below are the outputs; it crashed when we conducted testing using one thousand threads. Our application survived many failures, but after request number 1383, it crashed and went down. Below are the results,

No. of Threads	Throughput/min	Average/Ms	Deviation/Ms	Latency/Ms
100	2,526.848/minute	20	53	17.755
500	98.516/minute	336	428	394.511
1000	220.111/minute	304	426	144.9815
2000	-	-	-	-

CSV report generated from JMeter:

result_graph

Home Insert Page Layout Formulas Data Review View

Normal Page Break Preview Page Layout Custom Views Ruler Formula Bar Zoom 100% Unfreeze Panes Freeze Top Row Freeze First Column Split View Record Macros

C1149 HTTP Request LMS Home Page

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R
	timestamp	elapsed	label	responseCod	responseMe	threadName	dataType	success	failureMess	bytes	sentBytes	grpThreads	allThreads	URL	Latency	IdleTime	Connect	
1376	1.6674E+12	1138	HTTP Request LMS Home Page	200	OK	Thread Grou	text	TRUE		2550	119	228	228	http://localh	1138	0	0	
1377	1.6674E+12	1138	HTTP Request LMS Home Page	200	OK	Thread Grou	text	TRUE		2550	119	229	229	http://localh	1138	0	0	
1378	1.6674E+12	1133	HTTP Request LMS Home Page	200	OK	Thread Grou	text	TRUE		2550	119	229	229	http://localh	1133	0	0	
1379	1.6674E+12	7	HTTP Request LMS Home Page	200	OK	Thread Grou	text	TRUE		2550	119	230	230	http://localh	7	0	1	
1380	1.6674E+12	12	HTTP Request LMS Home Page	200	OK	Thread Grou	text	TRUE		2550	119	231	231	http://localh	12	0	1	
1381	1.6674E+12	1105	HTTP Request LMS Home Page	200	OK	Thread Grou	text	TRUE		2550	119	231	231	http://localh	1105	0	0	
1382	1.6674E+12	985	HTTP Request LMS Home Page	200	OK	Thread Grou	text	TRUE		2550	119	233	233	http://localh	985	0	0	
1383	1.6674E+12	984	HTTP Request LMS Home Page	200	OK	Thread Grou	text	TRUE		2550	119	233	233	http://localh	984	0	0	
1384	1.6674E+12	464	HTTP Request LMS Home Page	Non HTTP re	Non HTTP re	Thread Grou	text	FALSE		2437	0	370	370	http://localh	0	0	0	
1385	1.6674E+12	457	HTTP Request LMS Home Page	Non HTTP re	Non HTTP re	Thread Grou	text	FALSE		2437	0	370	370	http://localh	0	0	0	
1386	1.6674E+12	458	HTTP Request LMS Home Page	Non HTTP re	Non HTTP re	Thread Grou	text	FALSE		2437	0	370	370	http://localh	0	0	0	
1387	1.6674E+12	708	HTTP Request LMS Home Page	Non HTTP re	Non HTTP re	Thread Grou	text	FALSE		2437	0	368	368	http://localh	0	0	0	
1388	1.6674E+12	458	HTTP Request LMS Home Page	Non HTTP re	Non HTTP re	Thread Grou	text	FALSE		2437	0	368	368	http://localh	0	0	0	
1389	1.6674E+12	722	HTTP Request LMS Home Page	Non HTTP re	Non HTTP re	Thread Grou	text	FALSE		2437	0	367	367	http://localh	0	0	0	
1390	1.6674E+12	738	HTTP Request LMS Home Page	Non HTTP re	Non HTTP re	Thread Grou	text	FALSE		2437	0	367	367	http://localh	0	0	0	
1391	1.6674E+12	720	HTTP Request LMS Home Page	Non HTTP re	Non HTTP re	Thread Grou	text	FALSE		2437	0	367	367	http://localh	0	0	0	
1392	1.6674E+12	770	HTTP Request LMS Home Page	Non HTTP re	Non HTTP re	Thread Grou	text	FALSE		2437	0	365	365	http://localh	0	0	0	
1393	1.6674E+12	786	HTTP Request LMS Home Page	Non HTTP re	Non HTTP re	Thread Grou	text	FALSE		2437	0	365	365	http://localh	0	0	0	
1394	1.6674E+12	798	HTTP Request LMS Home Page	Non HTTP re	Non HTTP re	Thread Grou	text	FALSE		2437	0	364	364	http://localh	0	0	0	
1395	1.6674E+12	805	HTTP Request LMS Home Page	Non HTTP re	Non HTTP re	Thread Grou	text	FALSE		2437	0	364	364	http://localh	0	0	0	
1396	1.6674E+12	830	HTTP Request LMS Home Page	Non HTTP re	Non HTTP re	Thread Grou	text	FALSE		2437	0	364	364	http://localh	0	0	0	
1397	1.6674E+12	846	HTTP Request LMS Home Page	Non HTTP re	Non HTTP re	Thread Grou	text	FALSE		2437	0	363	363	http://localh	0	0	0	
1398	1.6674E+12	842	HTTP Request LMS Home Page	Non HTTP re	Non HTTP re	Thread Grou	text	FALSE		2437	0	363	363	http://localh	0	0	0	
1399	1.6674E+12	835	HTTP Request LMS Home Page	Non HTTP re	Non HTTP re	Thread Grou	text	FALSE		2437	0	363	363	http://localh	0	0	0	
1400	1.6674E+12	855	HTTP Request LMS Home Page	Non HTTP re	Non HTTP re	Thread Grou	text	FALSE		2437	0	362	362	http://localh	0	0	0	
1401	1.6674E+12	935	HTTP Request LMS Home Page	Non HTTP re	Non HTTP re	Thread Grou	text	FALSE		2437	0	362	362	http://localh	0	0	0	
1402	1.6674E+12	942	HTTP Request LMS Home Page	Non HTTP re	Non HTTP re	Thread Grou	text	FALSE		2437	0	362	362	http://localh	0	0	0	
1403	1.6674E+12	946	HTTP Request LMS Home Page	Non HTTP re	Non HTTP re	Thread Grou	text	FALSE		2437	0	360	360	http://localh	0	0	0	
1404	1.6674E+12	948	HTTP Request LMS Home Page	Non HTTP re	Non HTTP re	Thread Grou	text	FALSE		2437	0	360	360	http://localh	0	0	0	
1405	1.6674E+12	955	HTTP Request LMS Home Page	Non HTTP re	Non HTTP re	Thread Grou	text	FALSE		2437	0	359	359	http://localh	0	0	0	

Select destination and press ENTER or choose Paste

Graph view result:

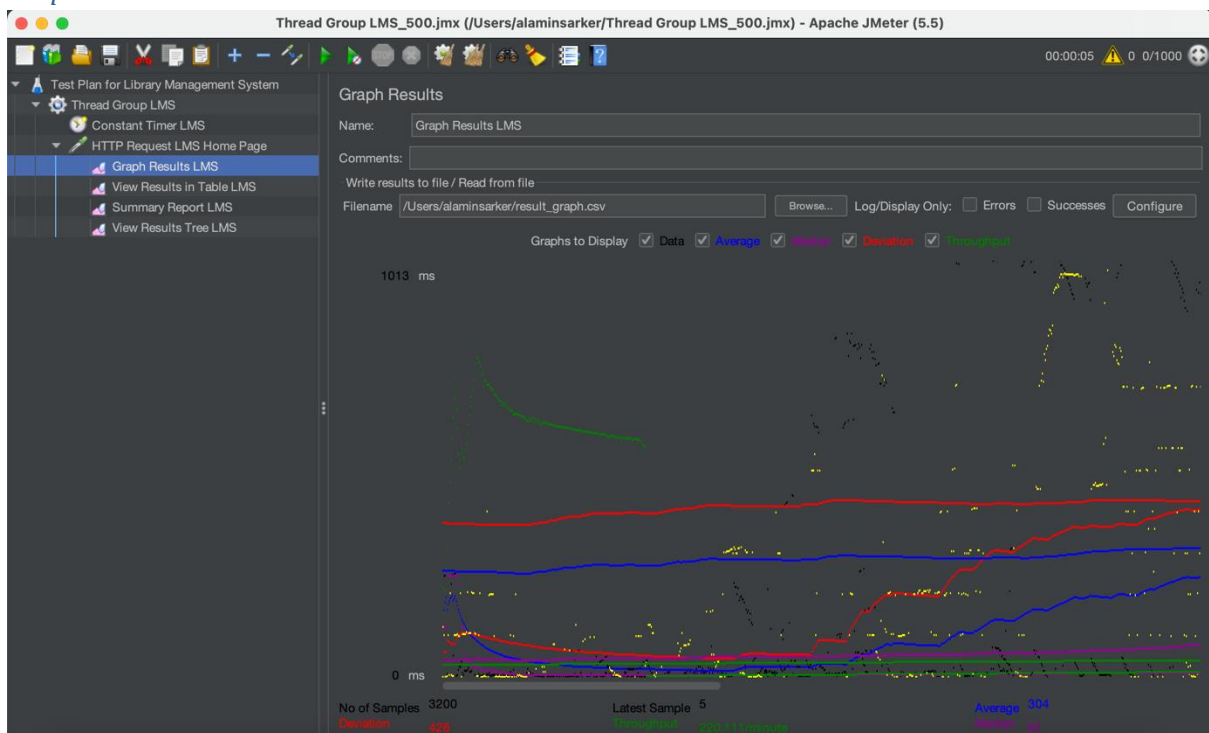


Table view result:

Thread Group LMS_500.jmx (/Users/alaminsarker/Thread Group LMS_500.jmx) - Apache JMeter (5.5)

00:00:05 0 0/1000

Test Plan for Library Management System

- Thread Group LMS
 - Constant Timer LMS
 - HTTP Request LMS Home Page
 - Graph Results LMS
 - View Results in Table LMS**
 - Summary Report LMS
 - View Results Tree LMS

View Results in Table

Name: View Results in Table LMS

Comments:

Write results to file / Read from file

Filename: /Users/alaminsarker/result_table.csv Browse... Log/Display Only: ☐ Errors ☐ Successes Configure

Sample #	Start Time	Thread Name	Label	Sample Tim...	Status	Bytes	Sent Bytes	Latency	Connect Ti...
3180	12:17:57.884	Thread Gro...	HTTP Requ...	18		2673	0	0	18
3181	12:17:57.885	Thread Gro...	HTTP Requ...	10		2673	0	0	10
3182	12:17:57.886	Thread Gro...	HTTP Requ...	14		2673	0	0	14
3183	12:17:57.887	Thread Gro...	HTTP Requ...	14		2673	0	0	14
3184	12:17:57.885	Thread Gro...	HTTP Requ...	20		2673	0	0	20
3185	12:17:57.887	Thread Gro...	HTTP Requ...	21		2673	0	0	20
3186	12:17:57.885	Thread Gro...	HTTP Requ...	24		2673	0	0	24
3187	12:17:57.902	Thread Gro...	HTTP Requ...	7		2673	0	0	7
3188	12:17:57.902	Thread Gro...	HTTP Requ...	8		2673	0	0	7
3189	12:17:57.884	Thread Gro...	HTTP Requ...	26		2673	0	0	26
3190	12:17:57.901	Thread Gro...	HTTP Requ...	9		2673	0	0	9
3191	12:17:57.905	Thread Gro...	HTTP Requ...	6		2673	0	0	6
3192	12:17:57.920	Thread Gro...	HTTP Requ...	4		2673	0	0	4
3193	12:17:57.920	Thread Gro...	HTTP Requ...	5		2673	0	0	5
3194	12:17:57.952	Thread Gro...	HTTP Requ...	2		2673	0	0	2
3195	12:17:57.952	Thread Gro...	HTTP Requ...	2		2673	0	0	2
3196	12:17:57.955	Thread Gro...	HTTP Requ...	1		2673	0	0	1
3197	12:17:57.955	Thread Gro...	HTTP Requ...	1		2673	0	0	1
3198	12:17:57.955	Thread Gro...	HTTP Requ...	1		2673	0	0	1
3199	12:17:57.959	Thread Gro...	HTTP Requ...	3		2673	0	0	3
3200	12:17:57.967	Thread Gro...	HTTP Requ...	5		2673	0	0	4

☒ Scroll automatically? ☐ Child samples? No of Samples: 3200 Latest Sample: 5 Average: 304 Deviation: 428

Summary report result:

Thread Group LMS_500.jmx (/Users/alaminsarker/Thread Group LMS_500.jmx) - Apache JMeter (5.5)

00:00:05 0 0/1000

Test Plan for Library Management System

- Thread Group LMS
 - Constant Timer LMS
 - HTTP Request LMS Home Page
 - Graph Results LMS
 - View Results in Table LMS
 - Summary Report LMS**
 - View Results Tree LMS

Summary Report

Name: Summary Report LMS

Comments:

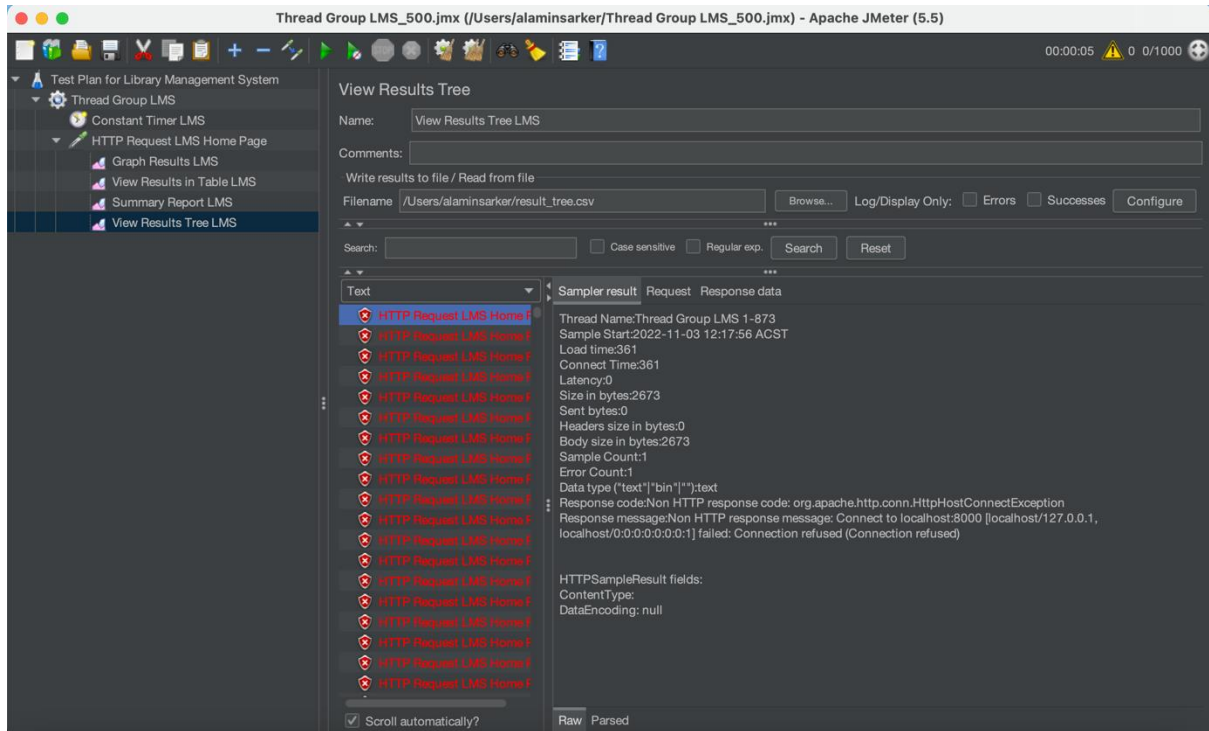
Write results to file / Read from file

Filename: /Users/alaminsarker/result_summary.csv Browse... Log/Display Only: ☐ Errors ☐ Successes Configure

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received K...	Sent KB/sec	Avg. Bytes
HTTP Requ...	3200	304	0	2328	426.12	41.41%	3.7/sec	9.07	0.25	2531.8
TOTAL	3200	304	0	2328	426.12	41.41%	3.7/sec	9.07	0.25	2531.8

☐ Include group name in label? Save Table Data ☒ Save Table Header

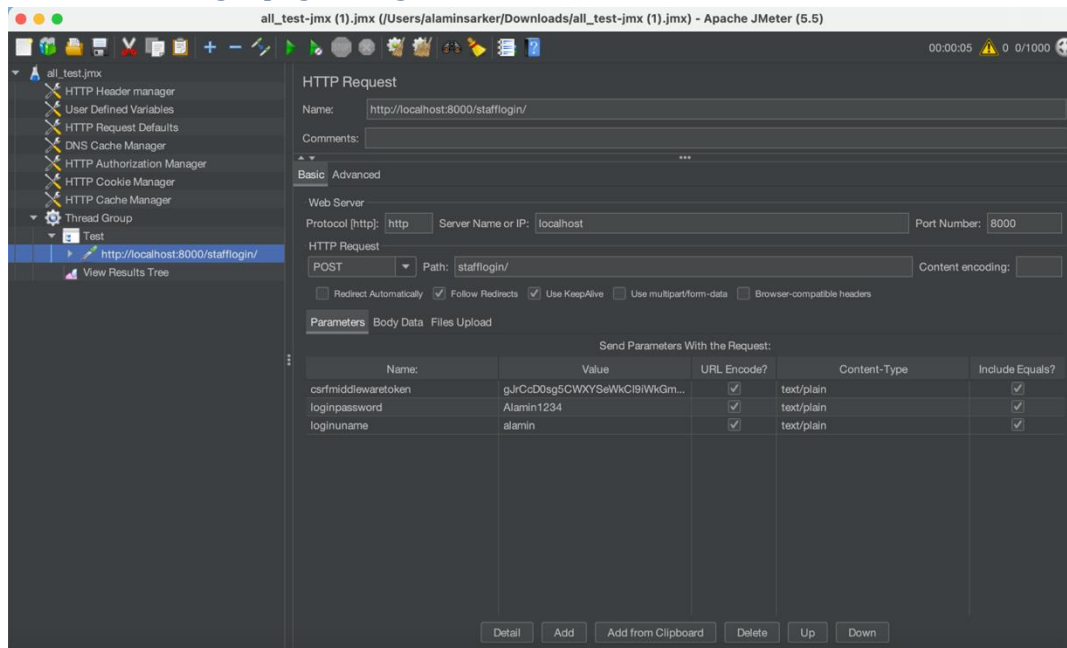
Tree View Result:



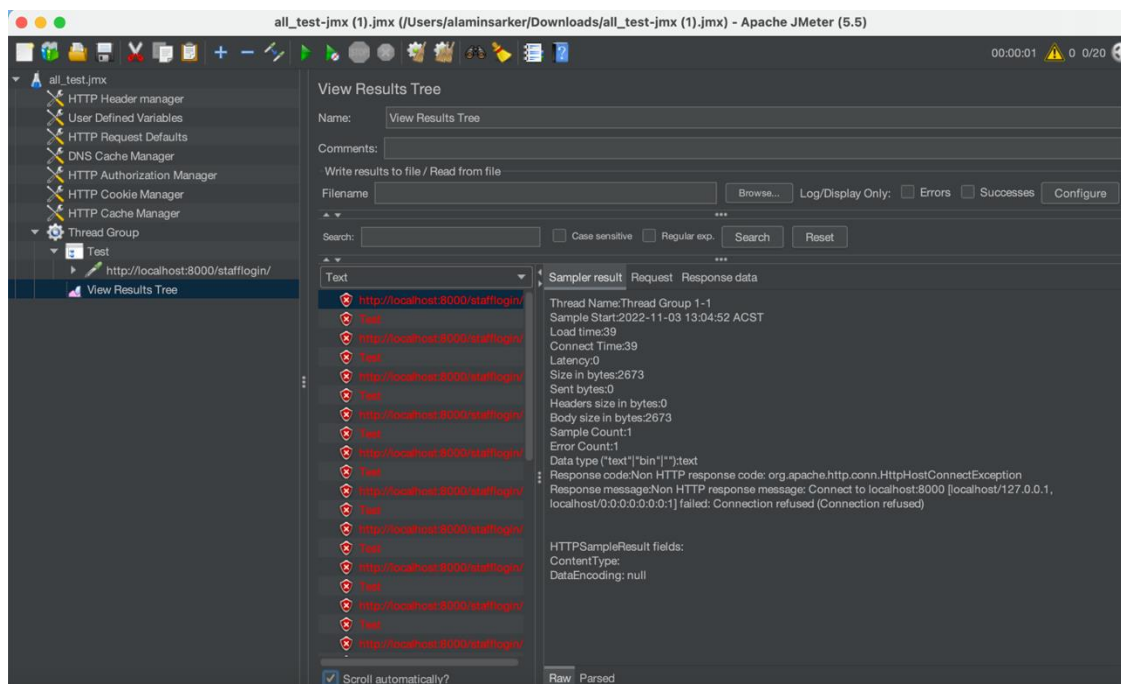
As we are running our application locally on a PC, it is not performing as expected. To make a better decision, we need to run our software on a server environment that can perform with the expected load.

We also tried to perform performance testing on other pages for our application. Still, we cannot perform tests on the login page using cookies because CSrfmiddlewaretoken is used, an advanced identification engine for fast, accurate, and reliable web app security and protects enterprise web applications from evolving cross-site requests forgery attacks. As a result, we must generate a different token for each request, but if we use cookies to login in through JMeter, it has a fixed token to log in. So, we cannot perform a load test on our login page or other pages requiring a login session.

Test for staff login page using cookies and cache:



Failed result:



Security Test:

To prevent the software from endangering the system and user data, software security testing verifies that the software is free of any potential flaws, hazards, or threats. Security testing proves that data and systems are secure and trustworthy and do not allow unauthorized inputs.

The fundamental objective of security testing is to determine the system's risks and assess any potential vulnerabilities so that threats can be encountered, and the system can continue to operate

without being compromised. Additionally, it aids in identifying any security risks that might be present in the design and enables programmers to solve issues.

Types of Software Security Testing:

Static Code Analysis:

Most developers begin their security testing using this method because it is the oldest. This test can be done manually, and engineers can read the code to look for security flaws.

Compliance Testing:

Software must comply with a client's established policies, and we ensure this by conducting compliance tests. We evaluate a piece of software in these tests by contrasting it with the actual configurations.

Penetration Testing:

This software testing uses simulation attacks to find the weak points in newly built software. A developer fixes the bugs in the codes once they are discovered.

Load Testing:

This test evaluates how well a piece of software operates under a heavy load. The reason for this test is Distributed-Denial-of-Service (DDoS) attack, which aims to interfere with an application's availability by flooding the host infrastructure with traffic or other requests.

SQL Injection Testing:

A SQL Injection test can be run for quote marks, apostrophes, brackets, and commas. Simple mistakes like these invite spammer attacks. SQL injection attacks are essential because they provide attackers access to the server database and allow them to obtain crucial data.

Software Security Testing tools:

Zed Attack Proxy (ZAP) is a multi-platform, open-source security testing tool for web applications developed by the Open Web Application Security Project (OWASP).

Key features of ZAP:

1. Automatic scanning
2. Easy to use
3. Multi-platform
4. Rest-based API
5. Support for authentication

Wfuzz: This tool's interface has no GUI and is developed using Python. The only drawback of this utility is that it can only be used via the command line.

Key features of Wfuzz:

1. Authentication support
2. Cookies fuzzing
3. Multi-threading
4. Multiple injection points
5. Support for proxy and SOCK

W3af: It is yet another acclaimed Python-built tool. This tool is ideal for web apps. W3af can find more than two hundred distinct kinds of security flaws.

Key features of W3af:

1. Blind SQL injection.
2. Buffer overflow.
3. Cross-site scripting.
4. Insecure DAV configurations.

Slap: It automates the discovery of a database vulnerability on a website and is completely free to use. SQL Map has a mighty testing engine to find different security threads.

Key features of SQL Map:

1. Automates the process of finding SQL injection vulnerabilities.
2. It can also be used for security testing on a website.
3. Robust detection engine.
4. Supports a range of databases, including MySQL, Oracle, and PostgreSQL.

Python Bandit: A source code security analysis tool called Bandit searches in Python programs for known vulnerabilities. It processes files to create an abstract syntax tree (AST), which is then used to run plugins against. It then produces a report on the results.

From the above options, we used Zed Attack Proxy (ZAP) to conduct security tests for our web application because it is an open-source security testing tool, particularly for web applications, and is both flexible and extensible.

Below are the security issues we found in our web application using ZAP:

We have two medium-priority alerts, four low-priority alerts, and one informational priority alert for our web application security. Let us investigate the security alerts.

1. Absence of Anti-CSRF Tokens: No known Anti-CSRF token was found in some HTML form. The search function is not implementing the form token to avoid cross-site scripting:

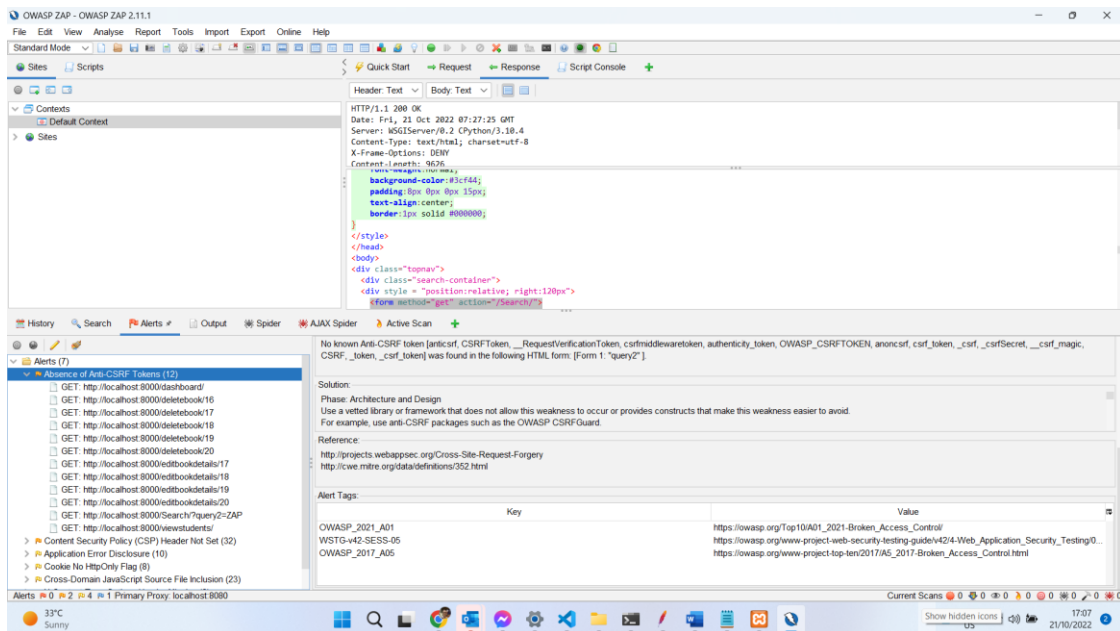


Figure 1: ZAP Test - CSRF Token Issue

Solution: Use a vetted library or framework that does not allow this weakness to occur or provides constructs that make this weakness easier to avoid. For example, we can use anti-CSRF packages such as the OWASP CSRFGuard.

2. Content Security Policy (CSP) Header Not Set:

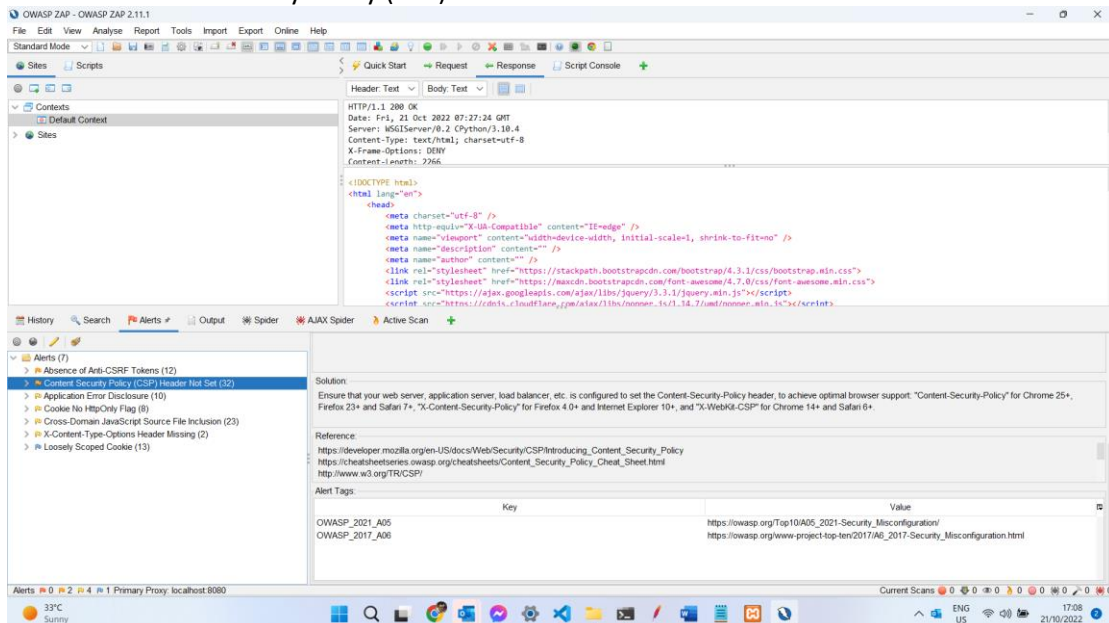


Figure 2: Content Security Policy (CSP) Header is not set

Solution: Ensuring our web server, application server, load balancer, etc., is configured to set the Content-Security-Policy header to achieve optimal browser support.

3. Application Error Disclosure: There are programming errors displayed on the page. We should have some error handling method in case of any error occurs.

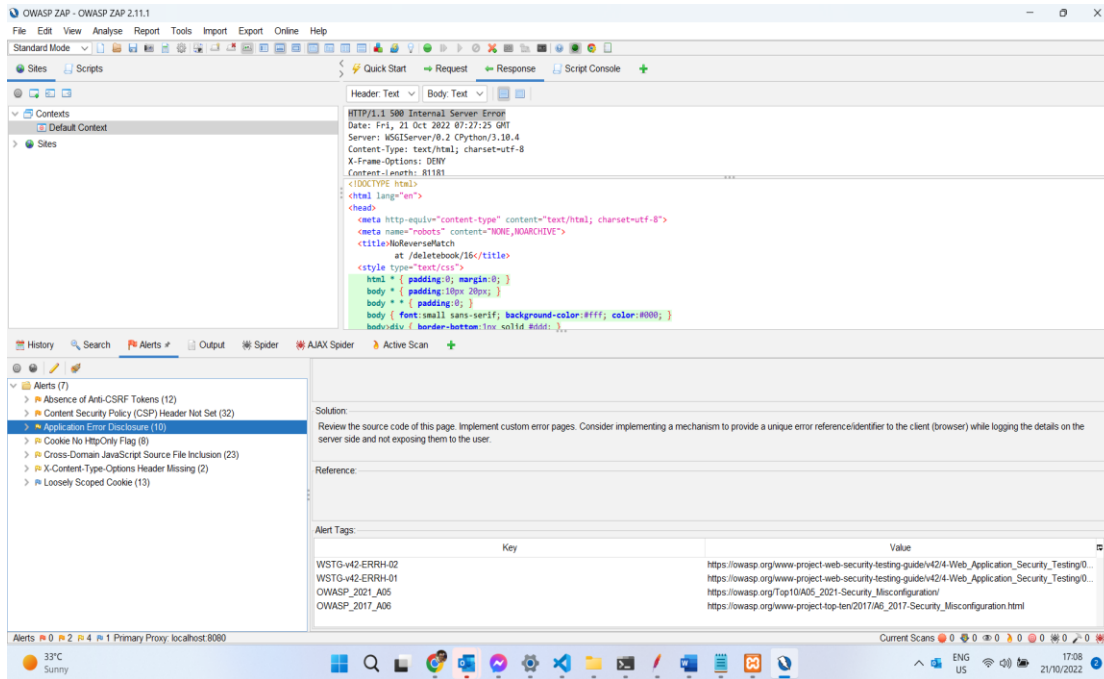


Figure 3: ZAP Test - Application Error Disclosure

Possible Solution: We can review the source code of this page and implement custom error pages. We can also consider implementing a mechanism to provide a unique error reference or identifier to the client (browser) while logging the details on the server side and not exposing them to the user.

1. Cookie Issue: No HTTP Only Flag

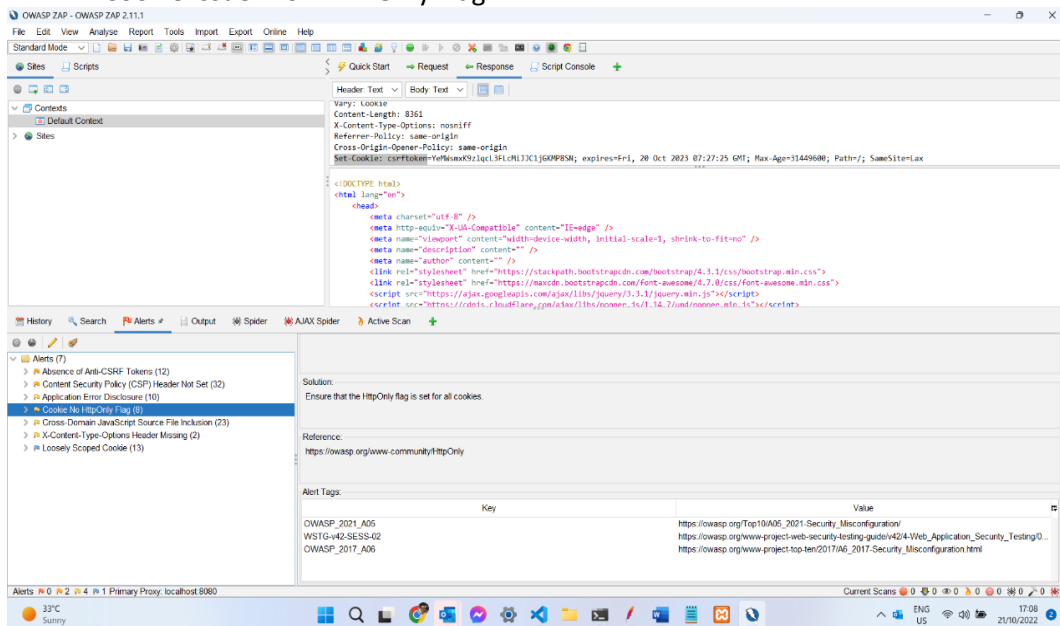


Figure 4: Cookie Warning

Possible Solution: Ensuring that the HTTP Only flag is set for all cookies.

1. Cross Domain JavaScript Sources file issue:

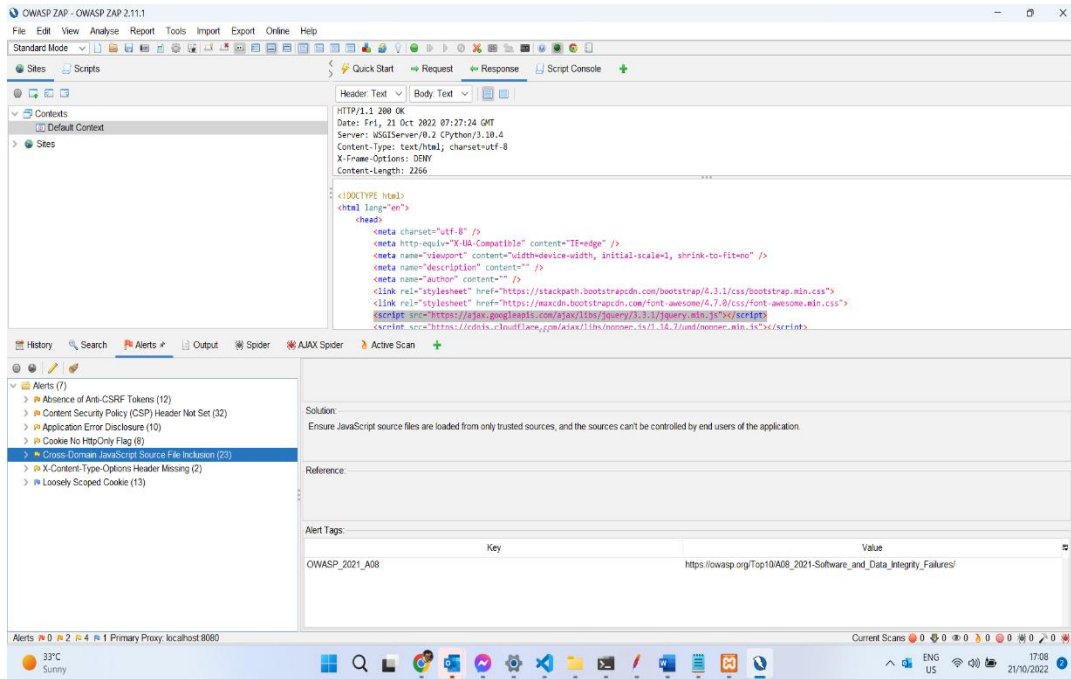


Figure 5: Cross-domain resources, including warning

Possible Solution: Ensuring JavaScript source files are loaded from only trusted sources, and end users of the application cannot control the sources.

1. X-Content-Type-Options header missing:

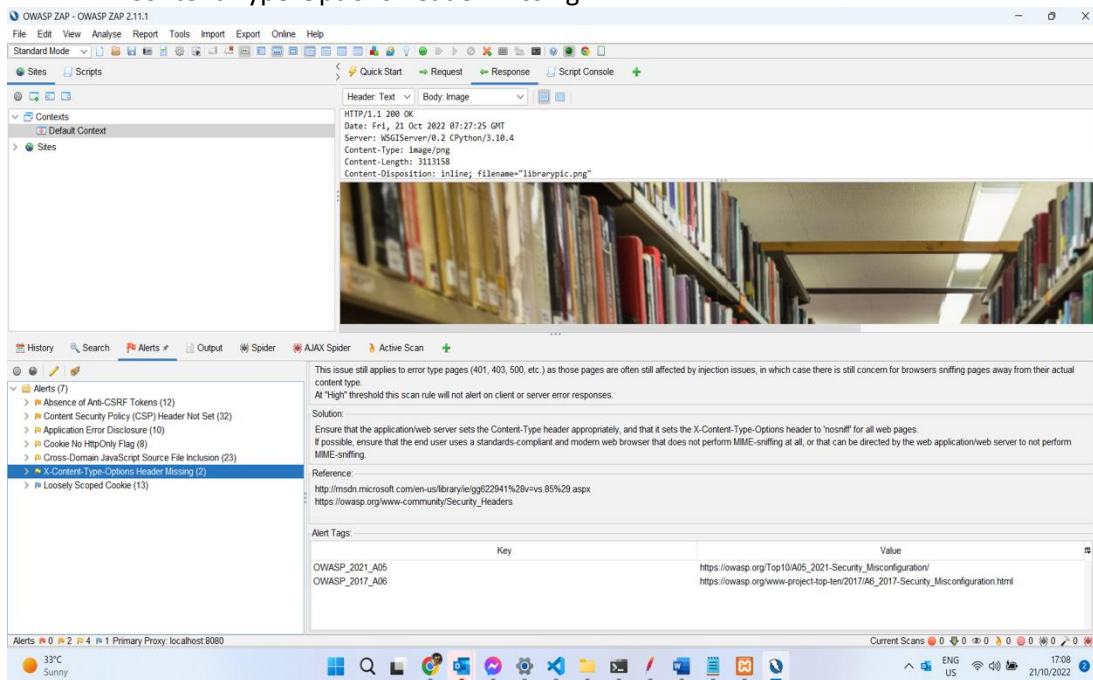


Figure 6: X-Content-Type-Options is missing

Possible Solution: Ensuring that the application or web server sets the Content-Type header appropriately and that it sets the X-Content-Type-Options header to 'no Sniff' for all web pages.

1. Cookie warning: Loosely scoped cookie because we used localhost as our domain for our web application.

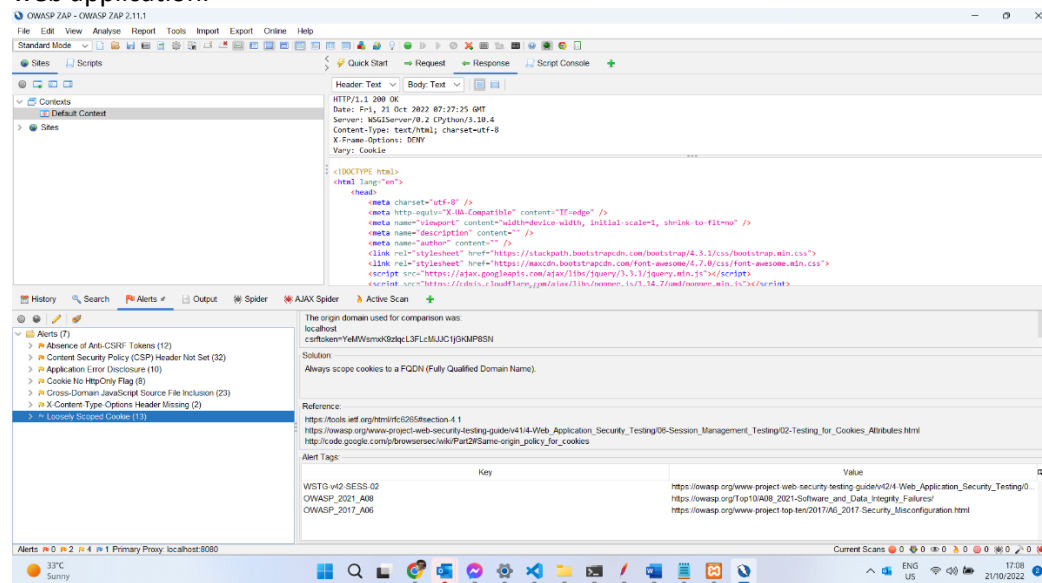


Figure 7: Loosely scoped cookie

Possible Solution: Always scope cookies to an FQDN (Fully Qualified Domain Name).

Functional Testing

Test plan:

Test strategy:

Various technology tools and frameworks are extensively supporting Test Automation in software development. With the appropriate technology and approach, QA teams can drastically reduce testing time and effort while maintaining comprehensive test coverage. Additionally, the LMS application has been released to the public with various features and tested manually. However, this would heavily cost most of the time for re-testing all features when having an updated version release. Thus, applying automated tests is essential not only to maintain a high standard of quality of the product but also to keep receiving valuable feedback from end users and product owners. Automated test optimizes the team's efforts and resources

Features to be tested:

Feature ID	Features	Descriptions
1	Home page	Application home page
2	Staff page	A staff entry page
3	Staff Sign up	Librarian staff user account signup page
4	Staff Login	Staff login page with registered user and password
5	Dashboard	Librarian user home page showing book list and

		its issuing status.
6	Add Book	Staff users can enter data for a new book
7	Add Student	Staff users can enter data for a new student
8	Issue Book	Staff users can make an issuing record for a book to a student
9	View Issued Book	View current issued book details
10	Search Book	Search books with inputted keywords.
11	View Student	View student details
12	Return Book	Staff users can input a return record associated with a fine that may cause a late return.
13	Log Out	Log the user out of the system.

Features not to be tested:

The entire Library Management System contains known issues in the latest released version. The functional testing is excluded from the unworked feature below, which has been notified to the tester team that this will not cause any blocking to other parts testing progress.

Test exclusive list: Contact us page and its related features.

Test deliverables and schedule:

The testing team to develop and deliver all important documents during the testing activity, and they will align with the work package and milestones below:

Work ID	Deliverable	Descriptions	Milestone
1	Test Plan Document	The testing plan is developed by the project starting and can be updated during the development phases.	Finished by week one
2	Test Cases Document + Manual test case + Automation test scripts	The test report documents automated test scripts and manual tests. Besides, a manual test is necessary when it needs humans for some testing.	Finished by week nine
3	Test Incident Reports Document	Execute test cases, and log bugs into the defect tracking system.	Finished by week ten
4	Test Summary Reports Documents	A system summary report includes fixed bugs and some carried-over bugs to subsequent upgrade version development phases.	Finished by week eleven

Responsibilities:

Student No	Member	Role	Responsibilities
------------	--------	------	------------------

1	Chao Yeo	QA Manager	Review test cases, review and approve the issues
2	Gia An Vong	QA Lead	Assigns tasks to track the testing progress
3	Ben Carruth	QA	Prepare test cases, and set up the test environment
4	Md Al Amin Sarker	QA	Setup performance and security tests, report the issues
5	Sandipkumar Bharatbhai Gangajaliya	QA	Develop automation test scripts, execute, and report the issues
6	Zhe Wei	QA	Develop automation test scripts, execute, and report the issues

Risks and mitigations:

The below shows the identified risks with descriptions and mitigations.

ID No.	R1
Risk	Delays in delivery of test items
Description	The developer team delivering test items later might require increased night shift scheduling to meet the delivery date.
Response	The developer and tester team work closely to keep progress informed. Communicate and negotiate with the project manager and product owner to adjust each sprint's schedule or release items.

ID No.	R2
Risk	Uncleared requirement
Description	An unclear requirement may highly cost both of developer and tester effort to fix the error.
Response	It is necessary to have communication between developers, testers, and stakeholders to have the feature developed as required.

ID No.	R3
Risk	Automated team member absent
Description	The unplanned absence of an automated tester may cause delays in product item delivery.
Response	Keep close communication and oversee team members' issues to have in-time support.

ID No.	R4
Risk	An automated team member lacks experience
Description	Less experience automated testers may cause delays in developing testing scripts.
Response	Develop a training strategy for project team members. Assign test scripts based on team member's skill level and feature complexity. Members should be required to work longer hours to ensure that the test items are delivered on time.

Comparison and justification of automation tool

Automation Testing necessitates purchasing new devices and software, which requires allocating time and resources to determine which tools/software are optimal.

Comparison of automated web testing tool:

	Robot Framework	Selenium
1	Test cases are easy to develop with Python	Test cases are easy to create with Python, java
1.	For Acceptance Test-Driven Development, the Robot framework is employed.	Selenium enables the programming of automation by the user.
2.	It incorporates a variety of libraries to facilitate automation.	It is a web driver without an integrated library.
3.	It can execute test cases without the need for external automation unwrapped. Test cases can be API or element based.	For executing selenium-written test cases, automation unwrapped is necessary.
4.	It is used to automate every sort of test, including acceptability testing and others.	Automation is limited to regression and performance testing only.
5.	Robot framework may be utilized to test both web and desktop apps.	Using selenium, web-based software and apps are tested.
6.	Users can define individualized test scenarios.	There is no custom test case functionality available.
7.	It may be utilized on all systems.	Selenium cannot operate on mobile devices platform
8.	Dictionary and list variables are supported.	It does not support variables.
9	The robot framework runs with Selenium web driver for web application testing. Test scripts are easy to develop with a natural language text base.	Developing Selenium test cases somehow requires experience with XPath access on a web application which is complex work for whom has less experience.

Those automated web testing tools have countless benefits and drawbacks. However, tool selection depends on several factors, such as the project's requirements, programming expertise, and budget. Our automated testing team has decided to use Robot Framework because of numerous factors:

1. The Library Management System is developed by Python
2. Most of the features are straightforward.
3. Robot Framework is one of our team experiences while working on CDU's projects.
4. Test scripts can be re-used from previous projects.
5. Testing timelines are short, with only three weeks.
6. A new member joins the team with less experience in another automated testing framework, and our team member can share their knowledge rapidly.
7. The automated scripts can run on cross-platform and cross-browser.

Environmental needs for Robot Framework:

1. Python 3.x
2. Robot Framework
3. Selenium Library
4. Browser: Chrome, Firefox, Edge, Opera, Safari (macOS)

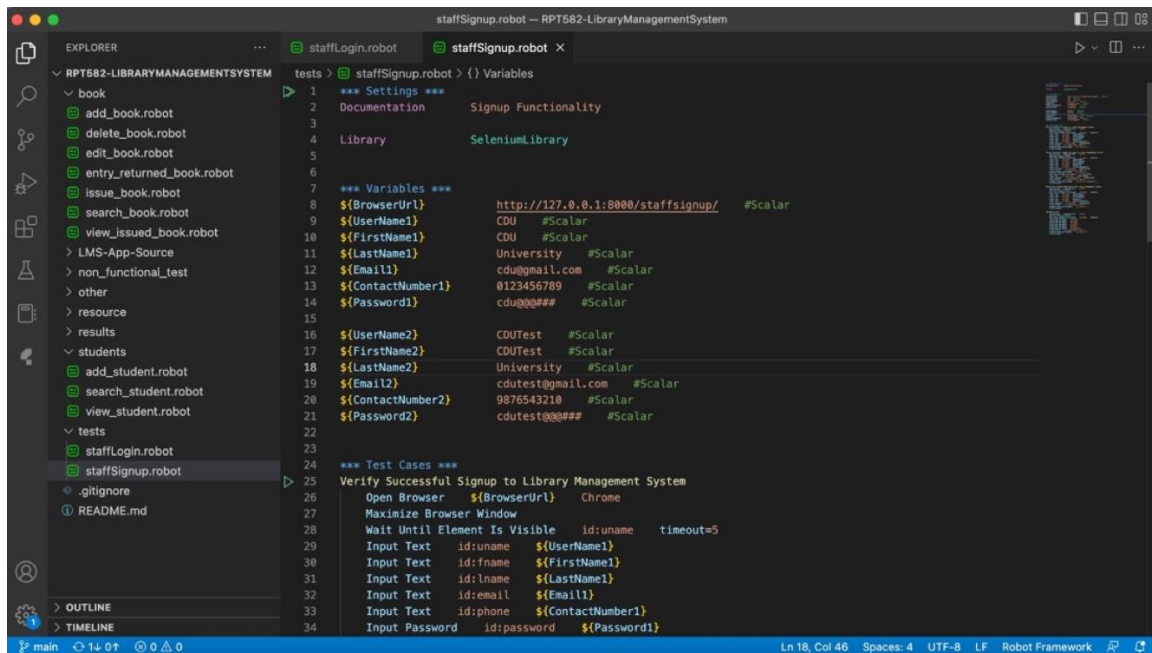
Test cases for functional requirements conducted by automated testing using Robot Framework:

Keywords used from Robot Framework and Selenium Library:

1. Open Browser
2. Maximize Browser Window
3. Wait Until Element Is Visible
4. Input Text
5. Input Password
6. Click Button
7. Element Should Be Visible
8. Close Browser
9. Should Not Be Empty
10. Chrome
11. Id
12. css:input[type='TYPE OF INPUT DECLARED IN CODE']
13. css:[href="/URL/"]

Allowing the staff to Signup (Create an account):

Writing automated testing code using Robot Framework:



Created four test cases for Staff signup:

1. Verify Successful Signup to Library Management System: test to create a new account.
2. Verify Successful Signup and Login to Library Management System: test if you can log in using the same credentials after creating a new account.
3. Verify User Already Registered with Library Management System: if already registered, the test should fail.
4. Validation Test

Executing robot file:

```

(base) Alamins-MacBook-Pro:tests alaminsarker$ robot -d results staffSignup.robot
staffSignup :: Signup Functionality
=====
Verify Successful Signup to Library Management System | PASS |
Verify Successful Signup And Login to Library Management System | PASS |
Verify User Already Registered with Library Management System | PASS |
Validation Test | PASS |
=====
staffSignup :: Signup Functionality
4 tests, 4 passed, 0 failed
=====
Output: /Users/alaminsarker/Documents/PRT582-LibraryManagementSystem/tests/results/output.xml
Log: /Users/alaminsarker/Documents/PRT582-LibraryManagementSystem/tests/results/log.html
Report: /Users/alaminsarker/Documents/PRT582-LibraryManagementSystem/tests/results/report.html

```

After running the code, we can see that all the test cases passed.

Results and reports:

28 minutes 48 seconds ago

LOG

Summary Information

Status:All tests passed

Documentation:Signup Functionality

Start Time:20221104 21:04:40.476

End Time:20221104 21:05:03.528

Elapsed Time:00:00:23.052

Log File:[log.html](#)

Test Statistics

Total Statistics

Total

Pass

Fail

Skip

Elapsed

Pass / Fail / Skip

All Tests

4

4

0

0

00:00:23

Statistics by Tag

Total

Pass

Fail

Skip

Elapsed

Pass / Fail / Skip

No Tags

Statistics by Suite

Total

Pass

Fail

Skip

Elapsed

Pass / Fail / Skip

staffSignup

4

4

0

0

00:00:23

Test Details

AllTagsSuitesSearch

Suite:

staffSignup

Status:4 tests total, 4 passed, 0 failed, 0 skipped

Documentation:Signup Functionality

Start / End Time:20221104 21:04:40.476 / 20221104 21:05:03.528

Elapsed Time:00:00:23.052

Log File:[log.html#s1](#)

Name

Documentation

Tags

Status

Message

Elapsed

Start / End

staffSignup_Verify Successful Signup to Library Management System

PASS

00:00:06.542

20221104 21:04:40.760

staffSignup_Verify Successful Signup And Login to Library Management System

PASS

00:00:06.787

20221104 21:04:47.302

staffSignup_Verify User Already Registered with Library Management System

PASS

00:00:05.225

20221104 21:04:54.073

staffSignup_Validation Test

PASS

00:00:04.224

20221104 21:05:03.524

Allowing the staff/librarian to log in:

Writing automated testing code using Robot Framework:

```
staffLogin.robot — RPT582-LibraryManagementSystem

tests > staffLogin.robot
1  *** Settings ***
2  Documentation      Login Functionality
3
4  Library            SeleniumLibrary
5
6
7  *** Variables ***
8  ${BrowserUrl}      http://127.0.0.1:8080/stafflogin/  #Scalar
9  ${UserName}         CDU  #Scalar
10 ${Password}         cdu@@##  #Scalar
11 ${WrongUserName}    Test  #Scalar
12 ${WrongPassword}   test@@##  #Scalar
13
14
15 *** Test Cases ***
16 Verify Successful Login to Library Management System
17   Open Browser    ${BrowserUrl}    Chrome
18   Maximize Browser Window
19   Wait Until Element Is Visible    id:loginname    timeout=5
20   Input Text      id:loginname     ${UserName}
21   Input Password  id:loginpassword ${Password}
22   Click Button    css:input[type='submit']
23   #Element Should Be Visible    css:[href="/dashboard/"]
24   Page should contain    Message : Successfully logged in
25   Close Browser
26
27 Invalid Credential Login Test Invalid User Password
28   Open Browser    ${BrowserUrl}    Chrome
29   Maximize Browser Window
30   Wait Until Element Is Visible    id:loginname    timeout=5
31   Input Text      id:loginname     ${WrongUserName}
32   Input Password  id:loginpassword ${WrongPassword}
33   Click Button    css:input[type='submit']
34   Element Should Be Visible    css:[href="/"]
```

We created eight test cases for Staff Login using all combinations of passwords:

1. Verify Successful Login to Library Management System
2. Failed Login Test Invalid Username and Invalid Password
3. Failed Login Test Invalid Username and Valid Password
4. Failed Login Test Valid Username and Invalid Password
5. Failed Login Test Empty Username and Valid Password
6. Failed Login Test Valid Username and Empty Password
7. Failed Login Test Empty Username and Empty Password
8. Validation Test

Executing robot file:

```
(base) Alamins-MacBook-Pro:tests alaminsarker$ robot -d results staffLogin.robot
=====
staffLogin :: Login Functionality
=====
Verify Successful Login to Library Management System | PASS |
-----
Failed Login Test Invalid User Name and Invalid Password | PASS |
-----
Failed Login Test Invalid User Name and Valid Password | PASS |
-----
Failed Login Test Valid User Name and Invalid Password | PASS |
-----
Failed Login Test Empty User Name and Valid Password | PASS |
-----
Failed Login Test Valid User Name and Empty Password | PASS |
-----
Failed Login Test Empty User Name and Empty Password | PASS |
-----
Validation Test | PASS |
-----
staffLogin :: Login Functionality | PASS |
=====
8 tests, 8 passed, 0 failed
=====
Output: /Users/alaminsarker/Documents/PRT582/RPT582-LibraryManagementSystem/tests/results/output.xml
Log: /Users/alaminsarker/Documents/PRT582/RPT582-LibraryManagementSystem/tests/results/log.html
Report: /Users/alaminsarker/Documents/PRT582/RPT582-LibraryManagementSystem/tests/results/report.html
(base) Alamins-MacBook-Pro:tests alaminsarker$
```

After running the code, we can see that all the test cases passed.

Results and reports:

staffLogin Report Generated: 20221104 23:34:54 UTC+09:30, 2 minutes 40 seconds ago

Summary Information

- Status: All tests passed
- Documentation: Login Functionality
- Start Time: 20221104 23:34:08.333
- End Time: 20221104 23:34:54.187
- Elapsed Time: 00:00:45.854
- Log File: log.html

Test Statistics

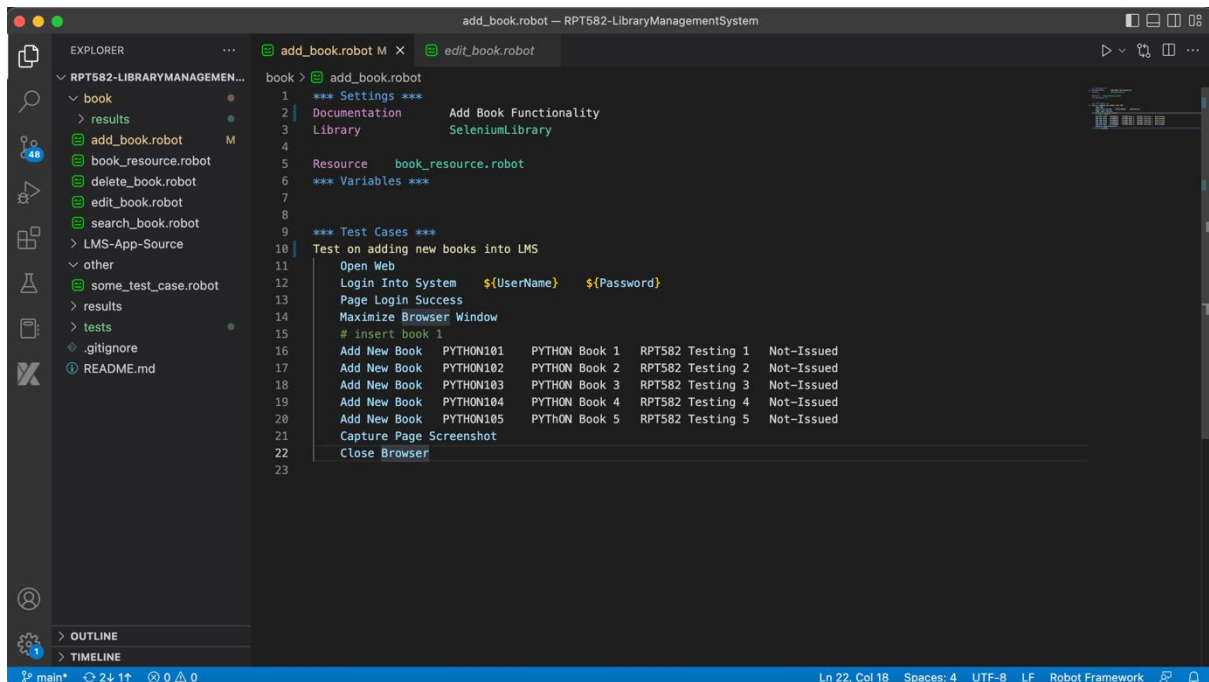
Total Statistics	Total	Pass	Fail	Skip	Elapsed	Pass / Fail / Skip
All Tests	8	8	0	0	00:00:45	

Test Details

Name	Documentation	Tags	Status	Message	Elapsed	Start / End
staffLogin.Failed Login Test Empty User Name and Empty Password			PASS		00:00:04.429	20221104 23:34:45.504
staffLogin.Failed Login Test Empty User Name and Valid Password			PASS		00:00:05.042	20221104 23:34:48.933
staffLogin.Failed Login Test Invalid User Name and Invalid Password			PASS		00:00:06.580	20221104 23:34:50.804
staffLogin.Failed Login Test Invalid User Name and Valid Password			PASS		00:00:05.623	20221104 23:34:16.347
staffLogin.Failed Login Test Valid User Name and Empty Password			PASS		00:00:04.655	20221104 23:34:25.927
staffLogin.Failed Login Test Valid User Name and Invalid Password			PASS		00:00:07.235	20221104 23:34:22.934
staffLogin.Validation Test			PASS		00:00:04.250	20221104 23:34:28.557
staffLogin.Verify Successful Login to Library Management System			PASS		00:00:07.683	20221104 23:34:40.847

Staff/librarian can add new books:

Writing automated testing code using Robot Framework:



We created a test case to check for adding new books functionality:

1. Test on adding new books into LMS (Learning Management System)

Executing robot file:

```

(base) Alamins-MacBook-Pro:book alaminsarker$ robot -d results add_book.robot
=====
Add Book :: Add Book Functionality
=====
Test on adding new books into LMS                                     | PASS |
-----
Add Book :: Add Book Functionality                                     | PASS |
1 test, 1 passed, 0 failed
=====
Output:  /Users/alaminsarker/Documents/PRT582/RPT582-LibraryManagementSystem/book/results/output.xml
Log:     /Users/alaminsarker/Documents/PRT582/RPT582-LibraryManagementSystem/book/results/log.html
Report:  /Users/alaminsarker/Documents/PRT582/RPT582-LibraryManagementSystem/book/results/report.html
(base) Alamins-MacBook-Pro:book alaminsarker$

```

After running the code, we see that the test case for adding new books passed.

Results and reports:

Add Book Report Generated 20221104 23:50:09 UTC+09:30 5 minutes 31 seconds ago LOG

Summary Information

Status: All tests passed
Documentation: Add Book Functionality
Start Time: 20221104 23:49:56.267
End Time: 20221104 23:50:09.133
Elapsed Time: 00:00:12.866
Log File: [log.html](#)

Test Statistics

Total Statistics	Total	Pass	Fail	Skip	Elapsed	Pass / Fail / Skip
All Tests	1	1	0	0	00:00:13	

Statistics by Tag	Total	Pass	Fail	Skip	Elapsed	Pass / Fail / Skip
No Tags						

Statistics by Suite	Total	Pass	Fail	Skip	Elapsed	Pass / Fail / Skip
Add Book	1	1	0	0	00:00:13	

Test Details

All Tags Suites Search

Status: 1 test total, 1 passed, 0 failed, 0 skipped
Total Time: 00:00:12.583

Name	Documentation	Tags	Status	Message	Elapsed	Start / End
Add Book: Test on adding new books into LMS			PASS		00:00:12.583	20221104 23:49:56.547 20221104 23:50:09.130

Staff/librarian can delete books:

Writing automated testing code using Robot Framework:

```
delete_book.robot
*** Settings ***
Documentation    Delete Book Functionality
Library         SeleniumLibrary

Resource        book_resource.robot
*** Variables ***
${book_id}     1

*** Test Cases ***
Verify Delete A Book Function
    Open Web
    Maximize Browser Window
    Login Into System    ${UserName}    ${Password}
    Page Login Success
    Capture Page Screenshot    delete-1-before.png
    # delete first row
    Click Element    xpath://tbody/tr[1]/td[6]/a[1]
    Capture Page Screenshot    after-1-before.png
    Close Browser

Verify Delete Another Book Function At Line Two
    Open Web
    Maximize Browser Window
    Login Into System    ${UserName}    ${Password}
    Page Login Success
    Capture Page Screenshot    delete-row-2-before.png
    # delete second row
    Click Element    xpath://tbody/tr[2]/td[6]/a[1]
    Capture Page Screenshot    delete-row-2-after.png
    Close Browser

Verify Delete Another Book Function At Line One
    Open Web
    Maximize Browser Window
```

We created three test cases for deleting books functionality:

1. Verify Delete a Book Function
2. Verify Delete Another Book Function at Line Two
3. Verify Delete Another Book Function at Line One

Executing robot file:

```
(base) Alamins-MacBook-Pro:book alaminsarker$ robot -d results delete_book.robot
=====
Delete Book :: Delete Book Functionality
=====
Verify Delete A Book Function                                     | PASS |
-----
Verify Delete Another Book Function At Line Two                 | PASS |
-----
Verify Delete Another Book Function At Line One                 | PASS |
-----
Delete Book :: Delete Book Functionality                         | PASS |
3 tests, 3 passed, 0 failed
=====
Output: /Users/alaminsarker/Documents/PRT582/RPT582-LibraryManagementSystem/book/results/output.xml
Log:    /Users/alaminsarker/Documents/PRT582/RPT582-LibraryManagementSystem/book/results/log.html
Report: /Users/alaminsarker/Documents/PRT582/RPT582-LibraryManagementSystem/book/results/report.html
(base) Alamins-MacBook-Pro:book alaminsarker$
```

After running the code, we see that all the test cases for deleting books passed.

Results and reports:

Delete Book Report Generated 20221105 00:07:34 UTC+09:30 5 minutes 47 seconds ago LOG

Summary Information

Status: All tests passed
Documentation: Delete Book Functionality
Start Time: 20221105 00:07:09.907
End Time: 20221105 00:07:34.187
Elapsed Time: 00:00:24.280
Log File: [log.html](#)

Test Statistics

Total Statistics	Total	Pass	Fail	Skip	Elapsed	Pass / Fail / Skip
All Tests	3	3	0	0	00:00:24	

No Tags

Statistics by Suite	Total	Pass	Fail	Skip	Elapsed	Pass / Fail / Skip
Delete Book	3	3	0	0	00:00:24	

Test Details

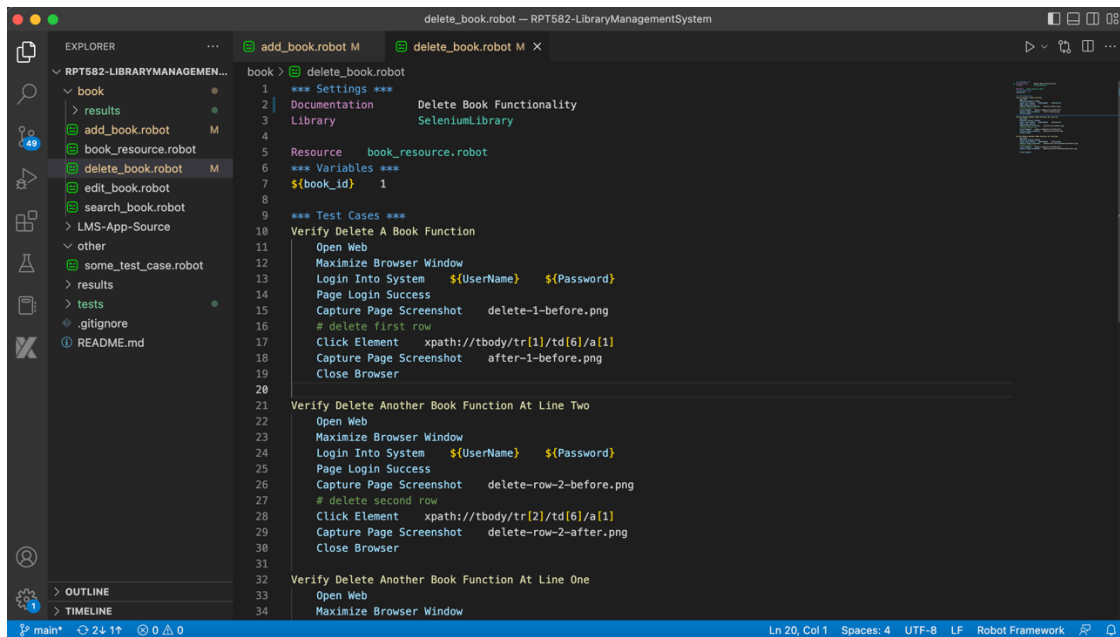
All Tags Suites Search

Status: 3 tests total, 3 passed, 0 failed, 0 skipped
Total Time: 00:00:23.931

Name	Documentation	Tags	Status	Message	Elapsed	Start / End
Delete Book, Verify Delete A Book Function			PASS		00:00:08.144	20221105 00:07:10.251 20221105 00:07:16.395
Delete Book, Verify Delete Another Book Function At Line One			PASS		00:00:08.532	20221105 00:07:25.652 20221105 00:07:34.184
Delete Book, Verify Delete Another Book Function At Line Two			PASS		00:00:07.255	20221105 00:07:18.396 20221105 00:07:25.651

Staff/librarian can edit or update books:

Writing automated testing code using Robot Framework:



We created three test cases for deleting books functionality:

1. Verify Edit a Book Function
2. Verify Edit a Second Book Function

Executing robot file:

```
(base) Alamins-MacBook-Pro:book alaminsarker$
(base) Alamins-MacBook-Pro:book alaminsarker$ robot -d results edit_book.robot
=====
Edit Book :: Edit Book Functionality
=====
Verify Edit A Book Function                                     | PASS |
-----
Verify Edit A Second Book Function                             | PASS |
-----
Edit Book :: Edit Book Functionality                             | PASS |
2 tests, 2 passed, 0 failed
=====
Output:  /Users/alaminsarker/Documents/PRT582/RPT582-LibraryManagementSystem/book/results/output.xml
Log:     /Users/alaminsarker/Documents/PRT582/RPT582-LibraryManagementSystem/book/results/log.html
Report:  /Users/alaminsarker/Documents/PRT582/RPT582-LibraryManagementSystem/book/results/report.html
(base) Alamins-MacBook-Pro:book alaminsarker$
```

After running the code, all the test cases for editing books passed.

Results and reports:

The screenshot shows a web browser displaying the 'Edit Book Report' page. The page has a green header and a white body. The browser address bar shows the URL: `RT582/RPT582-LibraryManagementSystem/book/results/report.html#totals`. The page title is 'Edit Book Report'. In the top right corner, it says 'Generated 20221105 00:19:28 UTC+09:30 27 seconds ago' and there is a 'LOG' button.

Summary Information

Status: All tests passed
Documentation: Edit Book Functionality
Start Time: 20221105 00:19:06.909
End Time: 20221105 00:19:28.156
Elapsed Time: 00:00:21.247
Log File: [log.html](#)

Test Statistics

Total Statistics	Total	Pass	Fail	Skip	Elapsed	Pass / Fail / Skip
All Tests	2	2	0	0	00:00:21	

Statistics by Tag

Total	Pass	Fail	Skip	Elapsed	Pass / Fail / Skip
No Tags					

Statistics by Suite

Total	Pass	Fail	Skip	Elapsed	Pass / Fail / Skip
Edit Book	2	2	0	0	00:00:21

Test Details

All Tags Suites Search

Status: 2 tests total, 2 passed, 0 failed, 0 skipped
Total Time: 00:00:20.811

Name	Documentation	Tags	Status	Message	Elapsed	Start / End
Edit Book_Verify Edit A Book Function			PASS		00:00:10.990	20221105 00:19:07.340 20221105 00:19:18.330
Edit Book_Verify Edit A Second Book Function			PASS		00:00:09.821	20221105 00:19:18.332 20221105 00:19:28.153

Staff/librarian can search for any book through book_Id:

Writing automated testing code using Robot Framework:

The screenshot shows the Robot Framework code editor with the following code:

```
search_book.robot — RPT582-LibraryManagementSystem

book > search_book.robot
1  *** Settings ***
2  Documentation    Search Book Functionality
3  Library          SeleniumLibrary
4
5  Resource         book_resource.robot
6  *** Variables ***
7  ${book_id}       1
8
9  *** Test Cases ***
10 Verify Search Book Function With ID test-1
11   Open Web
12   Login Into System    ${UserName}    ${Password}
13   Page Login Success
14   Search Book ID       PYTHON103
15   There are some book found
16   Capture Page Screenshot
17   Sleep                2s
18   Close Browser
19
20 Verify Search Book Function With ID test-2
21   Open Web
22   Login Into System    ${UserName}    ${Password}
23   Page Login Success
24   Search Book ID       PYTHON104
25   There are some book found
26   Capture Page Screenshot
27   Sleep                2s
28   Close Browser
29
```

We created two test cases for searching books functionality:

1. Verify Search Book Function with ID test one
2. Verify Search Book Function with ID test two

Executing robot file:

```
(base) Alamins-MacBook-Pro:book alaminsarker$ robot -d results search_book.robot
=====
Search Book :: Search Book Functionality
=====
Verify Search Book Function With ID test-1 | PASS |
-----
Verify Search Book Function With ID test-2 | PASS |
-----
Search Book :: Search Book Functionality | PASS |
2 tests, 2 passed, 0 failed
=====
Output: /Users/alaminsarker/Documents/PRT582/RPT582-LibraryManagementSystem/book/results/output.xml
Log: /Users/alaminsarker/Documents/PRT582/RPT582-LibraryManagementSystem/book/results/log.html
Report: /Users/alaminsarker/Documents/PRT582/RPT582-LibraryManagementSystem/book/results/report.html
(base) Alamins-MacBook-Pro:book alaminsarker$
```

After running the code, we see that all the test cases for searching books passed.

Results and reports:

Search Book Report Generated: 20221105 00:35:35 UTC+09:30, 10 seconds ago

Summary Information

Status: All tests passed
Documentation: Search Book Functionality
Start Time: 20221105 00:35:15.233
End Time: 20221105 00:35:35.336
Elapsed Time: 00:00:20.103
Log File: [log.html](#)

Test Statistics

Total Statistics	Total	Pass	Fail	Skip	Elapsed	Pass / Fail / Skip
All Tests	2	2	0	0	00:00:20	

Statistics by Tag

Total	Pass	Fail	Skip	Elapsed	Pass / Fail / Skip
No Tags					

Statistics by Suite

Total	Pass	Fail	Skip	Elapsed	Pass / Fail / Skip
Search Book	2	2	0	0	00:00:20

Test Details

Status: 2 tests total, 2 passed, 0 failed, 0 skipped
Total Time: 00:00:19.729

Name	Documentation	Tags	Status	Message	Elapsed	Start / End
Search Book, Verify Search Book Function With ID test-1			PASS		00:00:10.069	20221105 00:35:15.603 20221105 00:35:25.672
Search Book, Verify Search Book Function With ID test-2			PASS		00:00:09.660	20221105 00:35:25.674 20221105 00:35:35.334

Students can be added upon applications:

Writing automated testing code using Robot Framework:

```

@ add_student.robot X
students > @ add_student.robot > {} Settings
3 Library SeleniumLibrary
4 Resource ${CURDIR}${/}..\resource/resource.robot
5
6
7 Load in Interactive Console
8 *** Variables ***
9
10 *** Test Cases ***
11 Run | Debug | Run in Interactive Console
12 Verify Add New Student Function
13 Open Web
14 Login Into System ${UserName} ${Password}
15 Page Login Success
16 Maximize Browser Window
17 # insert student An Vong s354803
18 Add New Student An Vong s354803
19 Go To http://127.0.0.1:8000/viewstudents/
20 Page Should Contain An Vong
21 Page Should Contain s354803
22 Capture Page Screenshot
23 Close Browser
24 Run | Debug | Run in Interactive Console
25 Verify Add Second Student Function
26 Open Web
27 Login Into System ${UserName} ${Password}
28 Page Login Success
29 Maximize Browser Window
30 # insert student An Vong s354803
31 Add New Student MD AI Admin S361035
32 Go To http://127.0.0.1:8000/viewstudents/
33 Page Should Contain MD AI Admin
34 Page Should Contain S361035
35 Capture Page Screenshot
36 Close Browser
37 Run | Debug | Run in Interactive Console
38 Verify Add Third Student Function
39 Open Web
40 Login Into System ${UserName} ${Password}
41 Page Login Success
42 Maximize Browser Window
43 # insert student An Vong s354803
44 Add New Student Ben Carruth S587273
45 Go To http://127.0.0.1:8000/viewstudents/
46 Page Should Contain Ben Carruth
47 Page Should Contain S587273
48 Capture Page Screenshot
49 Close Browser

```

We created three test cases for adding student's functionality:

1. Verify Add New Student Function
2. Verify Add Second Student Function
3. Verify Add Third Student Function

Executing robot file:

```

=====
Add Student :: Add Students Functionality
=====
Verify Add New Student Function
DevTools listening on ws://127.0.0.1:54291/devtools/browser/93258100-db7a-4c66-807f-b0b1aa4ee389
Verify Add New Student Function | PASS |
-----
Verify Add Second Student Function
DevTools listening on ws://127.0.0.1:54320/devtools/browser/20e23165-58ce-423b-bf1d-de691efe8854
Verify Add Second Student Function | PASS |
-----
Verify Add Third Student Function
DevTools listening on ws://127.0.0.1:54346/devtools/browser/e497a508-7b50-4287-ad44-3371269f4574
Verify Add Third Student Function | PASS |
-----
Add Student :: Add Students Functionality | PASS |
=====
3 tests, 3 passed, 0 failed
=====
Output: C:\Program Files\Python310\Scripts\RPT582-LibraryManagementSystem-main\output.xml
Log: C:\Program Files\Python310\Scripts\RPT582-LibraryManagementSystem-main\log.html
Report: C:\Program Files\Python310\Scripts\RPT582-LibraryManagementSystem-main\report.html
PS C:\Program Files\Python310\Scripts\RPT582-LibraryManagementSystem-main>

```

After running the code, all the test cases for adding students passed.

Results and reports:

Add Student Report

Generated
20221106 21:28:15 UTC+09:30
36 seconds ago

Summary Information

Status: All tests passed

Documentation: Search Book Functionality

Start Time: 20221106 21:28:01.253

End Time: 20221106 21:28:35.841

Elapsed Time: 00:00:34.588

Log File: log.html

Test Statistics

Total Statistics

TotalPassFailSkipElapsedPass / Fail / Skip

All Tests330000:00:14

Statistics by Tag

TotalPassFailSkipElapsedPass / Fail / Skip

No Tags

Statistics by Suite

TotalPassFailSkipElapsedPass / Fail / Skip

Add Student330000:00:15

Test Details

AllTagsSuitesSearch

Status: 3 tests total, 3 passed, 0 failed, 0 skipped

Total Time: 00:00:14.413

NameDocumentationTagsStatusMessageElapsedStart / End

Add Student: Verify Add New Student Function20221106 21:28:01.41320221106 21:28:06.672PASS00:00:05.24920221106 21:28:01.41320221106 21:28:06.672

Add Student: Verify Add Second Student Function20221106 21:28:06.67420221106 21:28:11.249PASS00:00:04.57520221106 21:28:06.67420221106 21:28:11.249

Add Student: Verify Add Third Student Function20221106 21:28:11.25020221106 21:28:16.839PASS00:00:04.58920221106 21:28:11.25020221106 21:28:16.839

Students can issue books:

Writing automated testing code using Robot Framework:

```
issue_book.robot X
book > | issue_bookrobot > ...
  Run Suite | Debug Suite | Load in Interactive Console
1  *** Settings ***
2  Documentation      Issue Book Functionality
3  Library             SeleniumLibrary
4  Resource            ${CURDIR}${/}..\resource\resource.robot
5
  Load in Interactive Console
6  *** Variables ***
7
8
9  *** Test Cases ***
  Run | Debug | Run in Interactive Console
10 Verify Issue First Book
11   Open Web
12   Login Into System    ${UserName}    ${Password}
13   Page Login Success
14   Maximize Browser Window
15   Go To                http://127.0.0.1:8000/bookissue/
16   Input Text           id:book1        PYTHON101
17   Input Text           id:studentid     S357617
18   Click Element        //button[contains(text(),'Submit')]
19   Capture Page Screenshot
20   Close Browser
21
  Run | Debug | Run in Interactive Console
22 Verify Issue Second Book
23   Open Web
24   Login Into System    ${UserName}    ${Password}
25   Page Login Success
26   Maximize Browser Window
27   Go To                http://127.0.0.1:8000/bookissue/
28   Input Text           id:book1        PYTHON102
29   Input Text           id:studentid     s354803
30   Click Element        //button[contains(text(),'Submit')]
31   Capture Page Screenshot
32   Close Browser
33
  Run | Debug | Run in Interactive Console
34 Verify Issue Third Book
35   Open Web
36   Login Into System    ${UserName}    ${Password}
37   Page Login Success
38   Maximize Browser Window
39   Go To                http://127.0.0.1:8000/bookissue/
40   Input Text           id:book1        PYTHON103
41   Input Text           id:studentid     S361035
42   Click Element        //button[contains(text(),'Submit')]
43   Capture Page Screenshot
44   Close Browser
```

We created three test cases for issuing book functionality:

1. Verify Issue First Book
2. Verify Issue Second Book
3. Verify Issue Third Book

Executing robot file:

```
=====
Issue Book :: Issue Book Functionality
=====
Verify Issue First Book
DevTools listening on ws://127.0.0.1:62525/devtools/browser/c1466f10-8665-4a82-ba2e-f8de22b32e01
Verify Issue First Book                                     | PASS |
-----
Verify Issue Second Book
DevTools listening on ws://127.0.0.1:62558/devtools/browser/ebace7d5-ff95-452b-bc61-67e56d09edeb
Verify Issue Second Book                                   | PASS |
-----
Issue Book :: Issue Book Functionality                       | PASS |
3 tests, 3 passed, 0 failed
=====

Output:  C:\Program Files\Python310\Scripts\RPT582-LibraryManagementSystem-main\output.xml
Log:      C:\Program Files\Python310\Scripts\RPT582-LibraryManagementSystem-main\log.html
Report:   C:\Program Files\Python310\Scripts\RPT582-LibraryManagementSystem-main\report.html
=====
```

Results and reports:

Issue Book Report

Generated 2022/11/07 13:59:30 UTC+08:00
2 hours 46 minutes ago

Summary Information

Status: All tests passed

Documentation: Issue Book Functionality

Start Time: 2022/11/07 13:59:14.093

End Time: 2022/11/07 13:59:36.443

Elapsed Time: 00:00:16.350

Log File: [log.html](#)

Test Statistics

Total Statistics	Total	Pass	Fail	Skip	Elapsed	Pass / Fail / Skip
All Tests	3	3	0	0	00:00:16	

Statistics by Tag	Total	Pass	Fail	Skip	Elapsed	Pass / Fail / Skip
No Tags						

Statistics by Suite	Total	Pass	Fail	Skip	Elapsed	Pass / Fail / Skip
Issue Book	3	3	0	0	00:00:16	

Test Details

All Tags Suites Search

Status: 3 tests total, 3 passed, 0 failed, 0 skipped

Total Time: 00:00:16.175

Name	Documentation	Tags	Status	Message	Elapsed	Start / End
Issue Book: Verify Issue First Book			PASS		00:00:05.397	2022/11/07 13:59:14.264 - 2022/11/07 13:59:19.661
Issue Book: Verify Issue Second Book			PASS		00:00:06.323	2022/11/07 13:59:19.662 - 2022/11/07 13:59:24.985
Issue Book: Verify Issue Third Book			PASS		00:00:05.455	2022/11/07 13:59:24.985 - 2022/11/07 13:59:30.441

Issued books can be viewed with details:

Writing automated testing code using Robot Framework:

```
view_issued_book.robot X
book > view_issued_book.robot > ...
Run Suite | Debug Suite | Load in Interactive Console
1 *** Settings ***
2 Documentation      View Issue Book Functionality
3 Library             SeleniumLibrary
4 Resource            ${CURDIR}/${/}.\\resource/resource.robot
5
6
7 Load in Interactive Console
8 *** Variables ***
9
10 *** Test Cases ***
11 Run | Debug | Run in Interactive Console
12 Verify First Issued Book Of Student
13   Open Web
14   Login Into System    ${UserName}    ${Password}
15   Page Login Success
16   Maximize Browser Window
17
18   Go To    http://127.0.0.1:8000/viewissuedbook/
19   Page Should Contain    Zhe Wei
20   Page Should Contain    S357617
21   Page Should Contain    PYTHON Book 1
22   Page Should Contain    RPT582 Testing 1
23   Page Should Contain    7-11-2022
24   Page Should Contain    22-11-2022
25   Capture Page Screenshot
26   Close Browser
27 Run | Debug | Run in Interactive Console
28 Verify Second Issued Book Of Student
29   Open Web
30   Login Into System    ${UserName}    ${Password}
31   Page Login Success
32   Maximize Browser Window
33
34   Go To    http://127.0.0.1:8000/viewissuedbook/
35   Page Should Contain    An Vong
36   Page Should Contain    s354803
37   Page Should Contain    PYTHON Book 2
38   Page Should Contain    RPT582 Testing 2
39   Page Should Contain    7-11-2022
40   Page Should Contain    22-11-2022
41   Capture Page Screenshot
42   Close Browser
43 Run | Debug | Run in Interactive Console
44 Verify Third Issued Book Of Student
45   Open Web
46   Login Into System    ${UserName}    ${Password}
```

We created three test cases for viewing issued book functionality:

1. Verify the First Issued Book of the Student
2. Verify the Second Issued Book of the Student
3. Verify the Third Issued Book of the Student

Executing robot file:

```
=====
View Issued Book :: View Issue Book Functionality
=====
Verify First Issued Book Of Student
DevTools listening on ws://127.0.0.1:63813/devtools/browser/e37c455d-1fb4-4a23-84c9-1acf5ef8060a
Verify First Issued Book Of Student                                     | PASS |
-----
Verify Second Issued Book Of Student
DevTools listening on ws://127.0.0.1:63840/devtools/browser/6bd82bf2-7bdb-468b-b8f2-ad2ea7154553
Verify Second Issued Book Of Student                                     | PASS |
-----
View Issued Book :: View Issue Book Functionality                                     | PASS |
3 tests, 3 passed, 0 failed
=====
Output:  C:\Program Files\Python310\Scripts\RPT582-LibraryManagementSystem-main\output.xml
Log:     C:\Program Files\Python310\Scripts\RPT582-LibraryManagementSystem-main\log.html
Report:  C:\Program Files\Python310\Scripts\RPT582-LibraryManagementSystem-main\report.html
```

Results and reports:

View Issued Book Report

20221107 14:18:13 UTC+08:00
3 hours, 30 minutes ago

Summary Information

Status:

All tests passed

Documentation:

View Issue Book Functionality

Start Time:

20221107 14:17:59.399

End Time:

20221107 14:18:13.123

Elapsed Time:

00:00:13.724

Log File:

log.html

Test Statistics

Total Statistics

Total

Pass

Fail

Skip

Elapsed

Pass / Fail / Skip

All Tests

3

3

0

0

00:00:14

Statistics by Tag

Total

Pass

Fail

Skip

Elapsed

Pass / Fail / Skip

No Tags

Statistics by Suite

Total

Pass

Fail

Skip

Elapsed

Pass / Fail / Skip

View Issued Book

3

3

0

0

00:00:14

View Test Details

All

Tags

Suites

Search

Status:

3 tests total, 3 passed, 0 failed, 0 skipped

Total Time:

00:00:13.534

Name

Documentation

Tags

Status

Message

Elapsed

Start / End

ViewIssuedBook_VerifyFirstIssuedBookOfStudent

PASS

00:00:04.077

20221107 14:17:59.084
20221107 14:18:04.261

ViewIssuedBook_VerifySecondIssuedBookOfStudent

PASS

00:00:04.412

20221107 14:18:04.263
20221107 14:18:08.675

ViewIssuedBook_VerifyThirdIssuedBookOfStudent

PASS

00:00:04.445

20221107 14:18:08.677
20221107 14:18:13.123

Registered students can be viewed and searched:

Writing automated testing code using Robot Framework:

```
@ search_student.robot X
students > | search_student.robot > ...
Run Suite | Debug Suite | Load in Interactive Console
1  *** Settings ***
2  Documentation      Search Student Functionality
3  Library             SeleniumLibrary
4  Resource            ${CURDIR}/${/}..\resource/resource.robot
5
Load in Interactive Console
6  *** Variables ***
7
8
9  *** Test Cases ***
Run | Debug | Run in Interactive Console
10 View and search the first registered student
11   Open Web
12   Login Into System    ${UserName}    ${Password}
13   Page Login Success
14   Search Book ID       S357617
15   There are some book found
16   Sleep                2s
17   Go To                http://127.0.0.1:8000/viewstudents/
18   Page Should Contain  Zhe Wei
19   Capture Page Screenshot
20   Close Browser
21
Run | Debug | Run in Interactive Console
22 View and search the second registered student
23   Open Web
24   Login Into System    ${UserName}    ${Password}
25   Page Login Success
26   Search Book ID       s354803
27   There are some book found
28   Sleep                2s
29   Go To                http://127.0.0.1:8000/viewstudents/
30   Page Should Contain  An Vong
31   Capture Page Screenshot
32   Close Browser
33
Run | Debug | Run in Interactive Console
34 View and search the third registered student
35   Open Web
36   Login Into System    ${UserName}    ${Password}
37   Page Login Success
38   Search Book ID       S361035
39   There are some book found
40   Sleep                2s
41   Go To                http://127.0.0.1:8000/viewstudents/
42   Page Should Contain  MD Al Admin
43   Capture Page Screenshot
44   Close Browser
```

We created three test cases for searching student functionality:

1. View and search the first registered student
2. View and search the second registered student
3. View and search the third registered student

Executing robot file:

```
=====
Search Student :: Search Student Functionality
=====
View and search the first registered student
DevTools listening on ws://127.0.0.1:52591/devtools/browser/f336a36e-7700-4db5-b796-5194bdb1e58e
.....[21924:22972:1107/144230.052:ERROR:device_event_log_impl.cc(215)] [14:42:30.052] USB: usb_de
[21924:22972:1107/144230.054:ERROR:device_event_log_impl.cc(215)] [14:42:30.054] USB: usb_device_
View and search the first registered student                                     | PASS |
-----
View and search the second registered student
DevTools listening on ws://127.0.0.1:52622/devtools/browser/eda968de-03d6-4b00-89c6-18c350086a42
.....[20580:22076:1107/144237.011:ERROR:device_event_log_impl.cc(215)] [14:42:37.011] USB: usb_de
[20580:22076:1107/144237.013:ERROR:device_event_log_impl.cc(215)] [14:42:37.013] USB: usb_device_
View and search the second registered student                                   | PASS |
-----
View and search the third registered student
DevTools listening on ws://127.0.0.1:52652/devtools/browser/dba6b9c9-6095-4d73-8eba-fd8143a895f4
-----
Search Student :: Search Student Functionality                                     | PASS |
3 tests, 3 passed, 0 failed
=====
Output: C:\Program Files\Python310\Scripts\RPT582-LibraryManagementSystem-main\output.xml
Log:     C:\Program Files\Python310\Scripts\RPT582-LibraryManagementSystem-main\log.html
Report:  C:\Program Files\Python310\Scripts\RPT582-LibraryManagementSystem-main\report.html
```

Results and reports:

Search Student Report

20221107 14:42:40.17C:\RPT582
2 hours 18 minutes ago

Summary Information

Status: All tests passed

Documentation: Search Student Functionality

Start Time: 20221107 14:42:30.553

End Time: 20221107 14:42:40.204

Elapsed Time: 00:00:20.751

Log File: log.html

Test Statistics

Total Statistics	Total	Pass	Fail	Skip	Elapsed	Pass / Fail / Skip
All Tests	3	3	0	0	00:00:21	3 / 0 / 0

Statistics by Tag

No Tags

Statistics by Suite

Total	Pass	Fail	Skip	Elapsed	Pass / Fail / Skip
Search Student	3	3	0	0	00:00:21

Test Details

AllTagsSuitesSearch

Status: 3 tests total, 3 passed, 0 failed, 0 skipped

Total Time: 00:00:20.760

Name	Documentation	Tags	Status	Message	Elapsed	Start / End
Search Student: View and search the first registered student			PASS		00:00:07.101	20221107 14:42:30.742 / 20221107 14:42:32.843
Search Student: View and search the second registered student			PASS		00:00:05.930	20221107 14:42:32.844 / 20221107 14:42:39.772
Search Student: View and search the third registered student			PASS		00:00:05.529	20221107 14:42:39.764 / 20221107 14:42:40.203

Users can entry for any book that is returned:

Writing automated testing code using Robot Framework:

```
entry_returned_book.robot
book > entry_returned_book.robot > ...
Run Suite | Debug Suite | Load in Interactive Console
1 *** Settings ***
2 Documentation      Return Book Functionality
3 Library            SeleniumLibrary
4 Resource           ${CURDIR}/${/}..\resource/resource.robot
5
6 Load in Interactive Console
7 *** Variables ***
8
9 *** Test Cases ***
10 Run | Debug | Run in Interactive Console
11 Return and check the first returned book
12     Open Web
13     Login Into System    ${UserName}    ${Password}
14     Page Login Success
15     Maximize Browser Window
16     Go To                http://127.0.0.1:8000/returnbook/
17     Input Text           id:bookid2    PYTHON101
18     Click Element        //button[contains(text(),'Submit')]
19
20     Go To                http://127.0.0.1:8000/dashboard/
21     Page Should Contain  PYTHON Book 1
22     Page Should Contain  RPT582 Testing 1
23     Page Should Contain  Not-Issued
24     Capture Page Screenshot
25     Close Browser
26
27 Run | Debug | Run in Interactive Console
28 Return and check the second returned book
29     Open Web
30     Login Into System    ${UserName}    ${Password}
31     Page Login Success
32     Maximize Browser Window
33     Go To                http://127.0.0.1:8000/returnbook/
34     Input Text           id:bookid2    PYTHON102
35     Click Element        //button[contains(text(),'Submit')]
36
37     Go To                http://127.0.0.1:8000/dashboard/
38     Page Should Contain  PYTHON Book 2
39     Page Should Contain  RPT582 Testing 2
40     Page Should Contain  Not-Issued
41     Capture Page Screenshot
42     Close Browser
43
44 Run | Debug | Run in Interactive Console
45 Return and check the third returned book
46     Open Web
47     Login Into System    ${UserName}    ${Password}
48     Page Login Success
49     Maximize Browser Window
```

We created three test cases for returning book functionality:

1. Return and check the first returned book

2. Return and check the second returned book
3. Return and check the third returned book

Executing robot file:

```

=====
Entry Returned Book :: Return Book Functionality
=====
Return and check the first returned book
DevTools listening on ws://127.0.0.1:58672/devtools/browser/9e368b20-1833-4ae2-8ce2-20661744ac96
Return and check the first returned book                                     | PASS |
-----
Return and check the second returned book
DevTools listening on ws://127.0.0.1:58697/devtools/browser/35d708df-0ab2-4794-b040-1af4c91a773b
Return and check the second returned book                                   | PASS |
-----
Return and check the third returned book
DevTools listening on ws://127.0.0.1:58728/devtools/browser/d7df1f97-0259-469a-af88-a22a64473874
Return and check the third returned book                                   | PASS |
-----
Entry Returned Book :: Return Book Functionality                             | PASS |
3 tests, 3 passed, 0 failed
=====
Output: C:\Program Files\Python310\Scripts\RPT582-LibraryManagementSystem-main\output.xml
Log: C:\Program Files\Python310\Scripts\RPT582-LibraryManagementSystem-main\log.html
Report: C:\Program Files\Python310\Scripts\RPT582-LibraryManagementSystem-main\report.html
PS C:\Program Files\Python310\Scripts\RPT582-LibraryManagementSystem-main>

```

Results and reports:

Entry Returned Book Report

20221107 17:20:11 UTC+08:00

1 minute 36 seconds ago

Summary Information

Status:

All tests passed

Documentation:

Return Book Functionality

Start Time:

20221107 17:19:56.479

End Time:

20221107 17:20:11.005

Elapsed Time:

00:00:14.526

Log File:

log.html

Test Statistics

Total Statistics

Total

Pass

Fail

Skip

Elapsed

Pass / Fail / Skip

All Tests

3

3

0

0

00:00:14

Statistics by Tag

Total

Pass

Fail

Skip

Elapsed

Pass / Fail / Skip

No Tags

Statistics by Suite

Total

Pass

Fail

Skip

Elapsed

Pass / Fail / Skip

Entry Returned Book

3

3

0

0

00:00:15

Test Details

AllTagsSearch

Status:3 tests total, 3 passed, 0 failed, 0 skipped

Total Time:00:00:14.358

Name

Documentation

Tags

Status

Message

Elapsed

Start / End

Entry Returned Book: Return and check the first returned book

PASS

00:00:04.386

20221107 17:19:56.644

Entry Returned Book: Return and check the second returned book

PASS

00:00:04.690

20221107 17:20:01.530

Entry Returned Book: Return and check the third returned book

PASS

00:00:04.972

20221107 17:20:06.132

Conclusions

Ethical consideration:

The project was developed following ACS (Australian Computer Society) and IEEE/ethical ACM standards. There are similarities between the ACM and ACS Codes of Ethics; however, the ACM places more emphasis on cultural and societal contributions and the well-being of those concerned than the ACS. Taking all of this into account, our group's primary ethical concerns are:

1. Integrity: When developing User Requirements, it is essential to be upfront with customers and users about any limitations the product may have by design. Users should be able to rely on the product to deliver on their job's essential duties and be alerted to any delays or discrepancies in data. Furthermore, product quality is the foremost factor that brings customer success, which decides a high portion of customer satisfaction.
2. The Primacy of the Public Interest: Privacy regulations are most often linked to the development of databases and computer systems due to the importance of the public interest. Encryption should be

used for data transmission and storage. Besides, a tester with a high opportunity to handle most data should always be aware of personal privacy regulations.

3. **Competence:** The software's reliability and quality must meet or exceed the standard in the industry if customers are to integrate it confidently into their operations. The Software Development Team, including the Quality Control team, is incentivized to cut expenses wherever possible, minimizing the time spent on testing, routine updates, and routine maintenance. Sometimes unethical software companies rush to market with inferior goods because of time limitations. Because of that, as a software tester, it is crucial to report all known issues found during the testing phase.
4. **Professionalism:** The atmosphere and culture of the office should be conducive to employees' health and happiness. Headaches and eye fatigue pose severe health and safety risks and must be addressed expertly. Everyone on the team should be given the same level of support and encouraged to give 100% to achieve the most significant potential outcomes.
5. **Honesty:** the QC engineer must always be honest when reporting bugs that could endanger the lives of users of the system. This will assist the development team in resolving or mitigating the issue that may influence the organization's credit.

Further work:

There are limitations to Robot Framework-based automated testing, such as UI (User Interface) Testing, which require manual testing by QC engineers. In addition, there are upcoming enhancements that will allow JMeter to entirely execute performance tests, as the high-security level of the Django Framework prevented the auto bot from interacting with the system. In the following project, our team will investigate further performance testing techniques to ensure that our system is not vulnerable to hacking or assault.

Lesson learned:

The testing team has faced numerous challenges when executing automated tests on the Robot Framework. Every project, website, and page's HTML structure will have a unique design and level of complexity. Consequently, reusing Test-Script only partially alleviates the time constraints of generating new test scripts. In the future, it will be necessary to more precisely estimate the time required to construct automated test scripts for the web platform utilizing Robot Framework. In addition, to prevent impeding the project's progress, the testing team needs workshops in which team members can exchange their experiences and enhance their skills.

Conclusion:

In conclusion, in agile-based software development, it will be crucial to ensure product quality throughout all phases and phases. Consequently, the automation testing team will be essential in efficiently assessing product quality and minimizing expenses. This necessitates the usage of proxies; hence, software testing solutions such as Robot Framework are required. Even though there are still restrictions in the breadth of automated testing that prevent it from covering all features, software test scripts have helped solve a sizeable portion of the labour testers spend hours performing. In addition, the installation of numerous other add-on tools such as ZAP (Zed Attack Proxy) and JMeter has helped to evaluate the software's level of perfection and security, enabling the development team to improve security, anti-hacking, data safety, and user confidence.

Appendix

The following are all the result links to Non-Functional Testing compiled by Group One.